A sublinear one-sided tester for fixed-subgraph freeness in bounded-arboricity graphs

Talya Eden, Reut Levi, Dana Ron

arxiv:2404.18126v1; CC BY 4.0

Editorial note. Only thm:Gen-F-ub-intro is this task primary claim. Complete author manuscript body, revision macros, testing-model assumptions, all proofs, figures and references retained. Inputs/bbl mechanically expanded; original source archive and native files preserved separately.

Editorial note. Native dotless-i command inside a summation is rendered using its standard math-font glyph, preserving the original symbol; an unreferenced overview copy of the C6 algorithm step label receives an overview suffix so actual algorithm references remain unambiguous.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

We study the problem of testing C_k-freeness (k-cycle-freeness) for fixed constant $k > 3$ in graphs with bounded arboricity (but unbounded degrees). In particular, we are interested in one-sided error algorithms, so that they must detect a copy of C_k with high constant probability when the graph is ϵ-far from C_k-free.

We next state our results for constant arboricity and constant ϵ with a focus on the dependence on the number of graph vertices, n. The query complexity of all our algorithms grows polynomially with $1/\epsilon$.

1. As opposed to the case of $k = 3$, where the complexity of testing C_3-freeness grows with the arboricity of the graph but not with the size of the graph (Levi, ICALP 2021)[1] this is no longer the case already for $k = 4$. We show that $\Omega(n^{1/4})$ queries are necessary for testing C_4-freeness, and that $\widetilde{O}(n^{1/4})$ are sufficient. The same bounds hold for C_5.
2. For every fixed $k \geq 6$, any one-sided error algorithm for testing C_k-freeness must perform $\Omega(n^{1/3})$ queries.
3. For $k = 6$ we give a testing algorithm whose query complexity is $\widetilde{O}(n^{1/2})$.
4. For any fixed k, the query complexity of testing C_k-freeness is upper bounded by $O(n^{1-1/\lfloor k/2 \rfloor})$.

The latter upper bound builds on another result in which we show that for any fixed subgraph F, the query complexity of testing F-freeness is upper bounded by $O(n^{1-1/\ell(F)})$, where $\ell(F)$ is a parameter of F that is always upper bounded by the number of vertices in F (and in particular is $k/2$ in C_k for even k).

We extend some of our results to bounded (non-constant) arboricity, where in particular, we obtain sublinear upper bounds for all k.

Our $\Omega(n^{1/4})$ lower bound for testing C_4-freeness in constant arboricity graphs provides a negative answer to an open problem posed by (Goldreich, 2021).

[1] As presented in (Levi, ICALP 2021), the complexity of the algorithm depends on $\log \log n$, but this dependence can be replaced with at most a polylogarithmic dependence on the arboricity.

1 Introduction

Detecting small subgraphs with specific structures (referred to as finding network motifs) is a basic algorithmic task, with a variety of applications in biology, sociology and network science (see e.g. [21, 8, 31, 11, 7, 27, 5, 18, 6, 32, 29]). Of special interest is the natural case of subgraphs that are cycles of a fixed size k, which we denote by C_k. When the algorithm receives the entire graph as input, then by the well known result of Alon, Yuster and Zwick [4], this task can be solved in time $\widetilde{O}(n^\omega)$ where n is the number of graph vertices and ω is the exponent of matrix multiplication.[2] But what if we seek a sublinear-time (randomized) algorithm that does not read the entire graph? Namely, the algorithm is given query access to the graph[3] and should find a C_k when the graph is not C_k-free. This is clearly not possible if the graph contains only a single copy of C_k. However, is it possible to detect such a copy in sublinear-time when the graph is relatively far from being C_k-free? By "relatively far" we mean that it is necessary to remove a non-negligible fraction, denoted ϵ, of its edges in order to obtain an C_k-free graph. A closely related formulation of the question is whether we can design a one-sided error algorithm for testing C_k-freeness.[4]

If the maximum degree in the graph is upper bounded by a parameter $d_{\max}$, then the C_k-freeness testing problem can easily be solved by performing a number of queries that grows polynomially with $d_{\max}$ and exponentially with $\Theta(k)$ [20]. In particular, when $d_{\max} = O(1)$, then there is no dependence on the size of the graph G. We are however interested in considering graphs with varying degrees, so that, in particular, the maximum degree may be much larger than the average degree, and possibly as large as $\Theta(n)$.

For the special and interesting case where $k = 3$, i.e., the cycle is a triangle, Alon, Kaufman, Krivelevich and Ron [3] gave several upper and lower bounds on the query complexity of testing triangle-freeness as a function of the average degree d of the graph (in addition to the dependence on n and ϵ). While the upper and lower bounds are not tight in general, they are tight for $d = O(1)$, where the complexity is $\Theta(\sqrt{n})$ (for constant ϵ). The lower bound in this case is essentially based on "hiding" a small clique.

Since the aforementioned lower bound relies on the existence of a small dense subgraph, a natural question, studied by Levi [25], is whether it is possible to obtain improved (and possibly tight) results when the arboricity of the graph, denoted $arb(G)$, is bounded.[5] Focusing on the result under the assumption that $m \geq n$ (i.e., $d = \Omega(1)$) Levi showed that $\widetilde{O}(arb(G))$ queries are sufficient for testing triangle-freeness (the dependence on $1/\epsilon$ is polynomial), and that $\Omega(arb(G))$ queries are necessary.[6] In particular, when $arb(G)$ is a constant, the complexity is polynomial in $1/\epsilon$ and does not depend on the size of the graph.

In this work we seek to understand the complexity of testing C_k-freeness, in particular with one-sided error, for fixed $k > 3$. Our main focus is on constant arboricity graphs and some of our results extend to bounded arboricity graphs, as well as to F-freeness for any general subgraph F (of constant size). We note that the problem of testing cycle-freeness without requiring the cycle to be of specific length, is different from our problem. We further discuss this in Section 1.3. In the next subsection we state our findings.

1.1 Our results

Since our main focus is on graphs with constant arboricity, we first state our results in this setting, and later discuss our extensions to graphs with non-constant arboricity. Throughout this paper we assume that $m = \Omega(n)$ since even obtaining a single edge in the graph requires $\Omega(n/m)$ queries. Our algorithms use degree and neighbor queries and our lower bounds also allow pair queries (see Footnote 3). For simplicity, we state our results for constant ϵ. All our algorithms have a polynomial dependence on $1/\epsilon$, which is stated explicitly in the corresponding technical sections.

[2] The dependence on k is exponential, but k is considered a constant.

[3] The types of queries typically considered are neighbor queries ("what is the ith neighbor of a vertex v?"), degree queries ("what is the degree of a vertex v?"), and pair queries ("is there an edge between a pair of vertices v and u?").

[4] The problems are equivalent if the algorithm is not given access to degree queries, otherwise the algorithm might find evidence to the existence of a C_k without actually detecting one. We note that all our algorithms do detect copies of C_k when they reject.

[5] The arboricity of a graph G is the minimum number of forests required to cover its edges, and is equal (up to a factor of 2) to the maximum average degree of any subgraph of G.

[6] To be precise, this lower bound holds for $m \geq (arb(G))^3$ – if $m < (\arg(G))^3$ then the lower bound is $\Omega(m^{1/3})$. See also Footnote 1 regarding the upper bound.

Our first finding is that, as opposed to the case of $k = 3$, where the complexity of testing C_3-freeness grows with the arboricity of the graph but not with the size of the graph,[7] this is no longer the case for $k = 4$ (and larger k). In particular:

Theorem 1 The query complexity of one-sided error testing of C_4-freeness in constant-arboricity graphs over n vertices is $\widetilde{\Theta}(n^{1/4})$. The same bound holds for testing C_5-freeness.

The more detailed statements for testing C_4 and C_5 freeness can be found in Theorems 6–10.

Theorem 1 (together with the upper bound in [20]) answers negatively the following open problem raised by Goldreich.

Open problem (number 3.2 in [19]): From bounded degree to bounded arboricity.
Suppose that property Π is testable within complexity $Q(n, \epsilon)$ in the bounded-degree graph model. Provide an upper bound on the complexity of testing Π in the general graph model under the promise that the tested graph has constant arboricity. For example, can the latter complexity be linear in $Q(n, \epsilon)$ while permitting extra $poly(logn)$ or $1/\epsilon$ factors?

The $\Omega(n^{1/4})$ lower bound for testing C_4-freeness, answers this question negatively. Indeed, testing C_4-freeness in d-bounded degree graphs can be done with $poly(d, \epsilon)$ queries [20], while our lower bound suggest that even in constant arboricity graphs, a polynomial dependence on n is necessary.

When $k \geq 6$, we show that it is no longer possible to obtain a complexity of $\widetilde{O}(n^{1/4})$ as is the case for $k = 4, 5$.

Theorem 2 Let $k \geq 6$. Any one-sided error tester for the property of C_k-freeness in graphs of constant arboricity over n vertices must perform $\Omega(n^{1/3})$ queries.

While for C_6 we were not able to match the lower bound of $\Omega(n^{1/3})$, we were able to obtain a sublinear-time algorithm, as stated next.

Theorem 3 There exists a one-sided error algorithm for testing C_6-freeness in graphs of constant arboricity over n vertices whose query complexity is $\widetilde{O}(n^{1/2})$.

For general (fixed) k we prove the following upper bound.

Theorem 4 There exists a one-sided error algorithm for testing C_k-freeness in graphs of constant arboricity over n vertices whose query complexity is $O(n^{1-1/\lfloor k/2 \rfloor})$.

We also prove a more general result for testing F-freeness for any constant size subgraph F. Below, $\ell(F)$ is as defined in Definition 1, and is always upper bounded by the number of vertices in F.

Theorem 5 There exists a one-sided error algorithm for testing F-freeness in graphs of constant arboricity over n vertices whose query complexity is $O(n^{1-1/\ell(F)})$.

1.1.1 Extensions for general arboricity

We state our results for general arboricity graphs assuming that the algorithm is given an upper bound α on the arboricity of the graph (in the lower bounds the algorithm may be assumed to know the arboricity). Alternatively, if the algorithm receives as an input the number of edges, m, (as in previous results for C_k-freeness [3, 25]) instead of an upper bound on the arboricity, then we can estimate a notion known [25] as the "effective arboricity" of the graph, and depend on it instead of α. This is potentially beneficial since the effective arboricity can be much smaller than the actual arboricity of the graph, and it does not affect the asymptotic running times of our algorithms in terms of the dependence on the size of the graph and α. For further details see Section 2.

For C_4-freeness we give both an upper bound and a lower bound for general arboricity graphs. In particular, we show that a linear dependence on α is sufficient and a $\sqrt{\alpha}$-dependence is necessary (both for one-sided error algorithms) as stated next.

[7] We note that this is true also for other k-cliques for $k > 3$.

Theorem 6 There exists a one-sided error algorithm for testing C_4-freeness in graphs of arboricity at most α over n vertices whose query complexity is $\tilde{O}\left(\min\{n^{1/4}\alpha, \alpha + n^{3/4}\}/\epsilon^3\right)$.[8]

Theorem 7 Testing C_4-freeness with one-sided error in graphs over n vertices with arboricity $c_1 \log n < \alpha < n^{1/2}/c_1'$ for sufficiently large constants c_1 and c_1' requires $\Omega(n^{1/4}\alpha^{1/2})$ queries for constant ϵ.[9]

For general constant size subgraphs F (and in particular C_k) our upper bound also has at most a linear dependence on α – see Theorem 13 and Corollary 14 for a precise statement.

We comment that our lower bound of $\Omega(n^{1/3})$ for one-sided error algorithms, $k \geq 6$ and constant arboricity (stated in Theorem 2) also applies to graphs with non-constant arboricity (by adding a C_k-free subgraph with higher arboricity).[10]

We also note that it is possible to extend our algorithms for C_5 and C_6 freeness so as to get a polynomial (but not linear) dependence on α. However, these extensions do not introduce new techniques (and are most probably not optimal), so we do not present them here.

1.2 A high-level discussion of our algorithms and lower bounds

Before discussing each of our results in more detail, we highlight some common themes. The starting point of all our algorithms is that if a graph is ϵ-far from being C_k-free (for a constant k), then it contains $\Omega(\epsilon m)$ edge-disjoint cycles.[11] We next use the bounded arboricity of the graph. Specifically, if a graph has arboricity at most α, then the number of edges between pairs of vertices that both have degree greater than $\theta_0 = \Theta(\alpha/\epsilon)$, is at most $O(\epsilon m)$.

Hence, there is a set of edge-disjoint C_ks, which we denote by $\mathcal{C}$, such that $|\mathcal{C}| = \Omega(\epsilon m)$, and no C_k in $\mathcal{C}$ contains any edge between two vertices with degree greater than θ_0. In other words, for every k-cycle ρ in $\mathcal{C}$, and for every vertex v with degree greater than θ_0 in ρ, the two neighbors of v in ρ have degree at most θ_0. In particular, when α is a constant, the two neighbors have degree $O(1/\epsilon)$.

At this point our algorithms diverge, but there are two common aspects in the case of $k = 4, 5, 6$ which we would like to highlight. The first is that for the sake of "catching" one of the C_ks in $\mathcal{C}$, it will be useful to consider a subset, $\mathcal{C}'$, in which every vertex v that participates in one of the edge-disjoint C_ks in $\mathcal{C}'$ actually participate in $\Omega(\epsilon \cdot d(v))$ C_ks in $\mathcal{C}'$. The existence of such a subset follows by applying (as a mental experiment) a simple iterative process that removes C_ks with vertices that do not obey this constraint.

To illustrate why it is useful to have such a set $\mathcal{C}'$, consider the case of $k = 4$, and assume that a relatively large fraction of the C_4s in $\mathcal{C}'$ contain, in addition to the two vertices of degree at most $\theta_0 = O(\alpha/\epsilon)$, at least one other vertex that has degree at most $\theta_1 = O(n^{1/2}/\epsilon)$. In this case we can obtain such a vertex v with high probability (as discussed below), and then sample roughly $\sqrt{d(v)/\epsilon} = O(\sqrt{\theta_1/\epsilon}) = O(n^{1/4}/\epsilon)$ of its neighbors, so that the following holds. By (a slight variant of) the birthday paradox, with high constant probability we hit two of its neighbors, u and u', that reside on the same C_4 in $\mathcal{C}'$ (and hence have degree at most θ_0). By querying all the neighbors of u and u', we obtain this C_4.

However, what if for most of the C_4's in $\mathcal{C}'$ there are two vertices with degree significantly larger than $\sqrt{n}$ (that are "one opposite the other" on the C_4s)? Roughly speaking, in this case we exploit the fact that the number of such high-degree vertices is bounded, and we show how to detect a C_4 by performing random walks of length 2. A related issue arises in the case of $k = 6$, when there are three very high degree vertices on most C_6s in $\mathcal{C}'$. In this case we show how to essentially reduce the problem to testing triangle-freeness in a certain auxiliary graph. More precisely, the auxiliary graph is a multi-graph to which we have access only

[8] More precisely, for values $\alpha < \log n$, the complexity is $O(n^{1/4}\alpha^{1/2}\log^{1/2} n/\epsilon^3)$, for values $\log n < \alpha < \sqrt{n}$, the complexity is $O(n^{1/4}\alpha/\epsilon^3)$, and for values $\alpha > n^{1/2}$, it is $O((\alpha + n^{3/4})/\epsilon^3)$.

[9] Note that the two-sided error lower bound of $\Omega(n^{1/4})$ for constant arboricity graphs (as stated in Theorem 1) also holds for graphs with higher arboricity α, and in particular, $\alpha = O(\log n)$. This is the case since we can simply add a small subgraph with arboricity α and no C_4s to the lower bound construction.

[10] For an odd k, it suffices to add a dense bipartite graph, and for even k, by the Erdős girth conjecture [16], one can add a subgraph with arboricity $n^{2/k}$.

[11] To verify this, let G be a graph that is ϵ-far from being C_k-free for a fixed constant k. Consider any maximal set S of edge-disjoint k-cycles. Since by removing all $k \cdot |S|$ edges on these cycles, the graph can be made cycle-free, $|S| \geq \epsilon m/k = \Omega(\epsilon m)$.

to certain types of queries, so that we cannot apply the algorithm of [3]. However, we can still show how to obtain a triangle in this graph, and hence a C_6 in the original graph. Interestingly, our general lower bound of $\Omega(n^{1/3})$ for C_k-freeness, $k \geq 6$ builds on the lower bound for testing triangle-freeness of [3].

In the following subsections we assume for the sake of the exposition that ϵ is a constant.

1.2.1 The results for C_4-freeness (and C_5-freeness)

We discuss our results for C_4-freeness in graphs with general arboricity. The results for C_5-freeness in constant arboricity graphs are obtained using very similar techniques.

The algorithm. Our algorithm for testing C_4-freeness, Test-C_4-freeness, which has query complexity $\widetilde{O}(n^{1/4}\alpha)$, is governed by two thresholds: $\theta_0 = \Theta(\alpha)$, and $\theta_1 = \Theta(n^{1/2})$. For the sake of the current high-level presentation, we assume that[12] $\alpha \leq n^{1/2}$, so that $\theta_0 \leq \theta_1$.

The algorithm first samples $O(1)$ edges approximately uniformly by invoking a procedure Select-an-Edge,[13] and then randomly selects one of their endpoints. For each vertex v selected, it queries its degree, $d(v)$. If $d(v) \leq \theta_1$, then the algorithm selects $O(\sqrt{d(v)})$ random neighbors of v, and for each selected neighbor u such that $d(u) \leq \theta_0$, it queries all the neighbors of u. If $d(v) > \theta_1$, then the algorithm performs $\widetilde{O}(n^{1/4}\alpha^{1/2})$ random walks of length two from v. If a C_4 is observed in any one of these steps, then the algorithm rejects, otherwise it accepts.

The analysis of the algorithm. By the above description, the algorithm will only reject a graph if it detects a C_4, implying that it never errs on C_4-free graphs. Hence, consider a graph G that is far from being C_4-free. As discussed at the start of Section 1.2, the setting of $\theta_0 = \Theta(\alpha)$ (together with the fact that G is $\Omega(1)$-far from being C_4-free) implies the following. There exists a set, denoted $\mathcal{C}$, of $\Omega(m)$ edge-disjoint C_4s in G, such that no C_4 in $\mathcal{C}$ contains an edge between two vertices that both have degree greater than θ_0. Thus, for each C_4 in $\mathcal{C}$, there are at most two vertices with degree greater than θ_0, and they do not neighbor each other.

Considering the second aforementioned degree threshold θ_1 (and recalling that $\theta_1 \geq \theta_0$), we partition $\mathcal{C}$ into two subsets. The first, $\mathcal{C}_1$, consists of those C_4s in $\mathcal{C}$ that contain at most one vertex with degree greater than θ_1, and the second, $\mathcal{C}_2$, of the remaining C_4s in $\mathcal{C}$, which contain exactly two vertices with degree greater than θ_1. Since $\mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2$, at least one of these subsets is of size $\Omega(m)$.

C_4s with at most one high-degree vertex. Suppose first that $|\mathcal{C}_1| = \Omega(m)$. Observe that since each 4-cycle $\rho \in \mathcal{C}_1$ contains at least two vertices with degree at most θ_0 and at most one vertex with degree greater than θ_1, it must contain at least one vertex, with degree at most θ_1 whose neighbors on the C_4 both have degree at most θ_0. For an illustration, see the LHS of Figure 1. Furthermore, we show that there exists a subset of $\mathcal{C}_1$, which we denote by $\mathcal{C}'_1$, such that $|\mathcal{C}'_1| = \Omega(m)$, and every vertex v that participates in one of the C_4s in $\mathcal{C}'_1$, actually participates in $\Omega(d(v))$ edges-disjoint C_4s in $\mathcal{C}_1$. It follows that in this case, when the algorithm selects a random edge (almost uniformly), with high constant probability it will obtain an edge with (at least) one endpoint v having the above properties. Conditioned on the selection of such a vertex v, the algorithm selects $\Theta(\sqrt{d(v)})$ random neighbors of v. By the birthday paradox, with high constant probability, among these neighbors there will be a pair of vertices that reside, together with v, on a common C_4 in $\mathcal{C}'_1$. Once their (at most θ_0) neighbors are queried, this C_4 is revealed.

C_4s with two high-degree vertex. We now turn to the case in which $|\mathcal{C}_2| = \Omega(m)$. Here too we can show that there exists a subset of $\mathcal{C}_2$, denoted $\mathcal{C}'_2$, such that $|\mathcal{C}'_2| = \Omega(m)$, and every vertex v that participates in one of the C_4s in $\mathcal{C}'_2$ actually participates in $\Omega(d(v))$ edges-disjoint C_4s in $\mathcal{C}_2$.

Recall that by the definitions of $\mathcal{C}$ and $\mathcal{C}_2$ and since $\mathcal{C}'_2 \subseteq \mathcal{C}_2 \subseteq \mathcal{C}$, the following holds. For each 4-cycle ρ in $\mathcal{C}'_2$, since it is in $\mathcal{C}_2$, there are two vertices whose degree is greater than θ_1. Therefore, by the definition of

[12] Indeed, graphs with arboricity greater than $n^{1/2}$ necessarily contain at least one C_4, but since we are interested in a one-sided error algorithm, and α is only known to be an upper bound on α, the algorithm cannot reject if it is provided with $\alpha > n^{1/2}$.

[13] This is a fairly standard and simple procedure, where we use the fact that graph has bounded arboricity, so that most of its edges have at least one endpoint with degree θ_0.

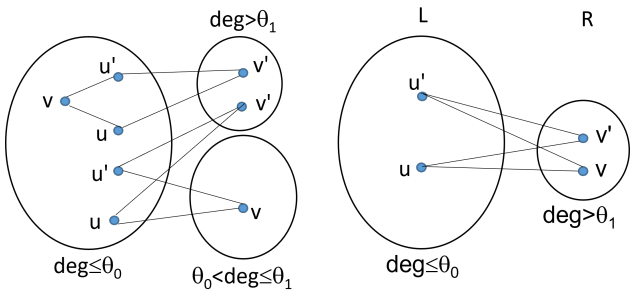


Figure 1: An illustration for some of the cases considered in the analysis of the algorithm for C_4-freeness. On the left side are two examples in which there is a single vertex v' with degree greater than θ_1, so that there is a vertex v with degree at most θ_1 with two neighbors whose degree is at most θ_0. On the right is an illustration when there are two such vertices with degree greater than θ_1.

$\mathcal{C}$, they are both adjacent on ρ to two vertices whose degree is at most θ_0. Hence, if we consider the subgraph induced by the vertices and edges of the C_4s in $\mathcal{C}'_2$, it is a bipartite graph, where on one side, denoted L, all vertices have degree at most θ_0, and on the other side, denoted R, all vertices have degree greater than θ_1. Furthermore, by the definition of $\mathcal{C}'_2$, for each vertex in R, a constant fraction of its neighbors (in the original graph G) belong to L, and for each vertex in L, a constant fraction of its neighbors belong to R. For an illustration, see the RHS of Figure 1.

Hence, if we select an edge almost uniformly and pick one of its endpoints with equal probability, with high constant probability we obtain a vertex $v \in R$. Conditioned on this event, since $d(v) > \theta_1$, the algorithm will perform $\widetilde{O}(n^{1/4}\alpha^{1/2})$ random walks of length two from v, and with high constant probability, a constant fraction of these walks will be of the form (v, u, v') where $u \in L$ and $v' \in R$. If for some v' we get two walks, (v, u, v') and (v, u', v') for $u \neq u'$, then a C_4 is detected.

Observe that since all vertices in R have degree greater than $\theta_1 = \Theta(n^{1/2})$, we have that $|R| \leq 2m/\theta_1 = O(n^{1/2}\alpha)$. This can be used to show that the expected number of pairs of walks that induce a C_4 is greater than 1. In order to show that we actually get such a pair with high constant probability, we perform a more careful analysis to bound the variance.

A (two-sided error) lower bound for testing C_4-freeness in constant arboricity graphs. To obtain this lower bound of $\Omega(n^{1/4})$, we define two distributions over graphs. In the support of the first distribution, $\mathcal{D}_0$, all graphs are C_4-free, and in the support of the second distribution, $\mathcal{D}_1$, all graphs are $\Omega(1)$-far from being C_4-free. Furthermore, $\mathcal{D}_0$ is uniform over all graphs isomorphic to a specific graph G_0, and $\mathcal{D}_1$ is uniform over all graphs isomorphic to a specific graph G_1.

We next describe a slightly simplified version of the two graphs (which cannot be used to prove the lower bound, but gives the essence of the proof). Both graphs are bipartite graphs, where one side, Y, contains $\Theta(\sqrt{n})$ vertices, and the other side, X, contains $\Theta(n)$ vertices, In G_0, each vertex in X has a unique pair of neighbors in Y (so there are no C_4s). On the other hand, in G_1, each vertex x in X has a "twin", x', where x and x' have the same pair of neighbors in Y (thus creating $\Omega(n)$ edge-disjoint C_4. See Figure 2. Observe that the arboricity of both graphs is 2 as for any subset of vertices S, the number of edges within S is at most $|S \cap X| \cdot 2$ so the average degree in the subgraph induced by S is at most 2.

In order to prove that no (possibly adaptive) algorithm can distinguish between a graph selected according to $\mathcal{D}_0$ and a graph selected according to $\mathcal{D}_1$, we define two processes, $\mathcal{P}_0$ and $\mathcal{P}_1$, which answer the queries of a testing algorithm while selecting a graph from $\mathcal{D}_0$ (respectively, $\mathcal{D}_1$) "on the fly". The lower bound of $\Omega(n^{1/4})$ follows from the fact that when performing fewer than $n^{1/4}/c$ queries (where c is a sufficiently large constant), for both distributions, with high constant probability, each new neighbor query is answered by a uniformly selected vertex id.

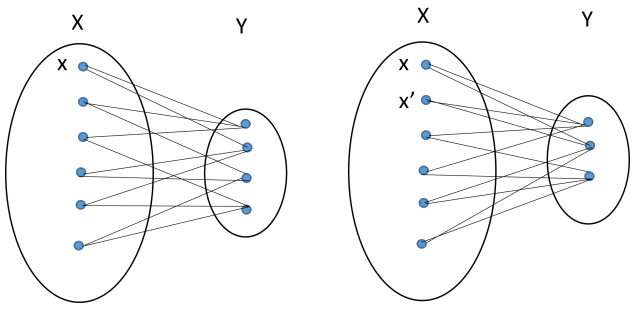


Figure 2: An illustration for the lower bound construction. The graph on the left is C_4-free while the graph on the right contains $\Omega(m)$ edge-disjoint C_4s and is hence $\Omega(1)$-far from being C_4-free.

A one-sided error lower bound for testing C_4-freeness in graphs with arboricity α. We next discuss the lower bound of $\Omega(n^{1/4}\alpha^{1/2})$ for graphs with arboricity $\alpha = \Omega(\log n)$ and one-sided error algorithms.

Here we define a single distribution $\mathcal{D}$ which is uniform over a family of graphs with arboricity α such that almost all graphs in this family are $\Omega(1)$-far from C_4-free.

Roughly speaking, the graphs in the support of $\mathcal{D}$ are random bipartite graphs, where one side, Y, is of size $\Theta(\sqrt{n}\alpha)$ and the other side, X, is of size $\Theta(n)$. Every vertex in X has α neighbors in Y, and every vertex in Y has $\Theta(\sqrt{n})$ neighbors in X. We need to show that if we select such a graph randomly, then on one hand it will be $\Omega(1)$-far from C_4-free, and on the other hand, in order to detect a C_4, any algorithm must perform $\Omega(n^{1/4}\alpha^{1/2})$ queries.

We next discuss the high-level idea as to why the resulting graphs are (with high constant probability) far from being C_4-free. Consider a fixed edge (x, y) in the bipartite graph, where $x \in X, y \in Y$. The number of C_4s this edge participates in is determined by the number of edges between the sets of neighbors of x and y, respectively $\Gamma(x)$ and $\Gamma(y)$. Recall that x has $\Theta(\alpha)$ neighbors and y has $\Theta(\sqrt{n})$ neighbors. Since overall there are $|X| \cdot |Y| = \Theta(n^{3/2}\alpha)$ potential pairs in the bipartite graph, and $\Theta(n\alpha)$ edges, each pair in $X \times Y$ is an edge with probability $\Theta(1/\sqrt{n})$. Hence, the expected number of edges between $\Gamma(x)$ and $\Gamma(y)$ is $|\Gamma(x)| \cdot |\Gamma(y)| \cdot (1/\sqrt{n}) = \Theta(\alpha)$. By analyzing the variance between pairs of edges, we furthermore show that with high constant probability, most edges do not participate in too many C_4s. Combining the two insights, it follows that with high constant probability, the graph is indeed far form being C_4-free.

In order to prove that any algorithm that performs at most $n^{1/4}\alpha^{1/2}/c$ queries (for a sufficiently large constant c), will not detect a C_4 with high constant probability, we actually prove that it will not detect any cycle. Roughly speaking, we show that by the randomness of the construction, since $|Y| = \Theta(\sqrt{n}\alpha)$, and the algorithm performs $O(\sqrt{|Y|})$ queries, each new neighbor query is answered by a uniformly distributed vertex that has not yet been observed. Therefore, the algorithm essentially views a forest.

A central challenge that we need to overcome is that we do not want to allow parallel edges, where the above construction might lead to their existence. One possibility is to first define the distribution over graphs with parallel edges and then to remove them. The benefit is that due to the higher degree of independence in the construction, it is somewhat easier to formally prove that the graphs obtained (with parallel edges) are with high probability $\Omega(1)$-far from C_4-free, and this remains the case when we remove parallel edges.

However, this creates a difficulty when we turn to argue that no (one-sided error) algorithm can detect a C_4 unless it makes $\Omega(n^{1/4}\alpha^{1/2})$ queries. The difficulty is due to the fact that in the formal proof we need to deal with dependencies that arise due to varying degrees (which occur because parallel edges are removed). While intuitively, varying degree should not actually "help" the algorithm, this intuition is difficult to formalize. Hence, we have chosen to define the distribution, from the start, over graphs that do not have parallel edges. This choice creates some technical challenges of its own (in particular in the argument that the graphs obtained are $\Omega(1)$-far from C_4-free), but we are able to overcome them. For the full details see Section B.

1.2.2 The algorithm for C_6-freeness

Recall that for C_6 we have a (one-sided error) testing algorithm whose query complexity is $\widetilde{O}(n^{1/2})$. In addition to assuming (for the sake of the exposition) that ϵ is a constant, we also ignore polylogarithmic factors in n. Similarly to the algorithm for testing C_4-freeness, the algorithm for testing C_6-freeness, Test-C_6-freeness, in constant arboricity graphs is governed by two thresholds. The first, θ_0, is of the order of the arboricity, so that it is a constant (recall that we assume that ϵ is a constant). The second, θ_2, is of the order of $\sqrt{n}$.

The algorithm repeats the following process several times. It selects a vertex v uniformly at random, and if $d(v) \leq \theta_0$, it performs a restricted BFS starting from v to depth 4. Specifically:

1. Whenever a vertex u is reached such that $d(u) \leq \theta_0$, all its neighbors are queried.
2. Whenever a vertex u is reached such that $d(u) > \theta_0$ and u is reached from a vertex u' such that $d(u') \leq \theta_0$, there are two sub-cases. If $d(u) \leq \theta_1$, then all of u's neighbors are queried. Otherwise, θ_1 neighbors of u are selected uniformly at random.
3. Whenever a vertex u is reached from a vertex u' such that both $d(u) > \theta_0$ and $d(u') > \theta_0$, the BFS does not continue from u.

The algorithm rejects if and only if it observes a C_6.

Consider a graph that is far from being C_6-free, so that it contains a set of $\Omega(m) = \Omega(n)$ edge-disjoint C_6s. Furthermore, it contains such a set, denoted $\mathcal{C}$ for which every C_6 in $\mathcal{C}$ contains at most three vertices with degree greater than θ_0, and furthermore, these vertices are not adjacent on the C_6. We partition $\mathcal{C}$ into three subsets: $\mathcal{C}_1$, $\mathcal{C}_2$, and $\mathcal{C}_3$, depending on the number of vertices with degree greater than θ_0 that it contains.

If either $|\mathcal{C}_1| = \Omega(m)$, or $|\mathcal{C}_2| = \Omega(m)$, then it is not hard to show that the algorithm will detect a C_6 with high constant probability. The more interesting part of the proof is handling the case in which only $|\mathcal{C}_3| = \Omega(m)$.

In this case we define an auxiliary multi-graph, denoted G', over the set of vertices that participate in C_6s belonging to $\mathcal{C}_3$, and have degree greater than θ_0 (in G). We denote this set of vertices by M, and the set of vertices with degree at most θ_0 that participate in these C_6s, by L.

Assume for simplicity that each vertex in L has degree exactly 2 (i.e., it participates in a single C_6). For each pair of vertices in M, we put in G' a set of parallel edges, whose size equals the number of length-2 paths between them in G that pass through vertices in L. Hence, for each C_6 in $\mathcal{C}$, we have a triangle in G', where these triangles are edge-disjoint, and we denote their set by $\mathcal{T}$. See Figure 3.

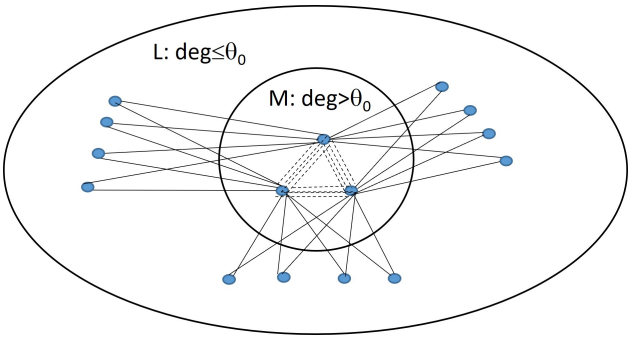


Figure 3: An illustration of the auxiliary (multi-)graph G' in the C_6-freeness testing algorithm. The dashed lines represent edges in G', each one corresponding to a length-2 path in G that passes through a vertex with degree at most θ_0.

Observe that selecting a vertex uniformly at random from L and querying its two neighbors in M corresponds to selecting an edge uniformly at random in G'. If we add an additional simplifying assumption by which (in G), vertices belonging to M only neighbor vertices belonging to L, then our algorithm on G

essentially translates to picking a random edge in G'. Then depending on the degree of the endpoints, either querying all their neighbors in G' or θ_1 random neighbors.

Let H denote the subset of vertices in M whose degree in G is greater than θ_1. If relatively many triangles in $\mathcal{T}$ contain at most one vertex in H, then we are done, since these triangles contain an edge for which both endpoints have degree at most θ_1. Hence, it remains to address the case in which almost all triangles in $\mathcal{T}$ have two or three vertices in H.

Roughly speaking, in this case we show that the existence of many edge-disjoint, but not vertex-disjoint, triangles in G' that contain such high-degree vertices implies the existence of "many more" triangles that may be caught by our algorithm. As an illustrative extreme (but easy) special case, assume that in G' there are only three vertices. Then the existence of some number t of edge-disjoint triangles between them, actually implies the existence of t^3 (non edge-disjoint) triangles.

1.2.3 The general lower bound for C_k-freeness, $k \geq 6$

We establish our general lower bound of $\Omega(n^{1/3})$ for one-sided error testing of C_k-freeness when $k \geq 6$ by building on a lower bound for testing triangle-freeness that appears in [3, Lemma 2]. This lower bound on testing triangle-freeness is based on the difficulty of detecting a triangle in graphs selected uniformly from a family $\mathcal{G}_{n'}$ of graphs in which almost all graphs are $\Omega(1)$-far from being triangle-free. All graphs in the family are d-regular tri-partite graphs over n' vertices and the lower bound on the number of queries necessary to detect a triangle (with constant probability), is $\Omega(\min\{d, n'/d\})$. By setting $d = \sqrt{n'}$, the lower bound is $\Omega(\sqrt{n'})$.

We show that, for any constant $k \geq 6$, if we had a one-sided error testing algorithm $\mathcal{A}$ for C_k-freeness of graphs with n vertices and constant arboricity using at most $n^{1/3}/c$ queries (for a constant c), then we would be able to detect triangles in graphs selected uniformly from $\mathcal{G}_{n'}$ using at most $\sqrt{n'}/c'$ queries (for a constant c').

To this end we define an algorithm $\mathcal{A}$ that, given query access to a graph $G' \in \mathcal{G}_{n'}$, implicitly defines a graph G for which the following holds. First, the number of vertices in G is $n = \Theta((n')^{3/2})$, and the number of edges is $m = \Theta(m')$, where m' is the number of edges in G' (so that $m' = \Theta((n')^{3/2})$). Second, G has arboricity 2. Third, the distance of G to C_k-freeness is of the same order as the distance of G' to triangle-freeness. Fourth, there is a one-to-one correspondence between triangles in G' and C_ks in G. The basic idea is to replace edges in the tri-partite graph G' with paths of length $k/3$. See Figure 4

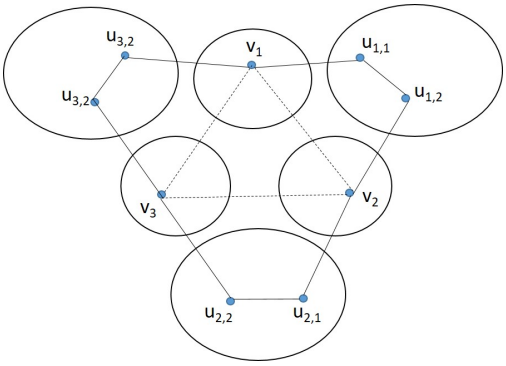


Figure 4: An illustration for the lower bound construction for C_k-freeness in constant arboricity graphs when $k = 9$. The three circles in the middle and the dashed lines represent a graph $G' \in \mathcal{G}_{n'}$. The outer circles represent the additional vertices in G. Since $k = 9$ in this example, each edge in G' is replaced by a path of length 3 in G.

Assuming there existed a testing algorithm $\mathcal{A}$ as stated above, the algorithm $\mathcal{A}'$ would use it to try and find a C_k in G (and hence a triangle in G'). In order to be able to run $\mathcal{A}$ on G, the algorithm $\mathcal{A}'$ must be able to answer queries of $\mathcal{A}$ to G by performing queries to G'. We show how this can be done with a constant

multiplicative overhead. Hence, the lower bound of $\Omega(\sqrt{n'})$ for testing triangle-freeness (when the degree is $\Theta(\sqrt{n'})$) translated into a lower bound of $\Omega(n^{1/3})$ for testing C_k-freeness.

1.2.4 The general upper bound for C_k-freeness

Recall that our starting point is that if G is $\Omega(1)$-far from being C_k-free, then it contains a set $\mathcal{C}$ of $\Omega(m)$ edge-disjoint C_k's that do not contain any edge between vertices that both have degree greater than $\theta_0 = \Theta(\alpha)$. We refer to vertices with degree at most θ_0 as light vertices, and to those with degree greater than θ_0 as heavy. Hence, each C_k in $\mathcal{C}$ has at least $\lceil k/2 \rceil$ light vertices, and each heavy vertex on it neighbors two light vertices.

We present two different algorithms, where each of them is suitable for a different setting. The basic idea of both algorithms is to take a large enough sample of vertices and edges so that the subgraph determined by the sampled light vertices and their incident edges, as well as the sampled edges, contains a copy of C_k. The query complexity of each algorithm is stated following its high-level description.

The first algorithm. Our first algorithm simply samples vertices uniformly, independently at random, and then performs queries that reveal the neighbors of all light vertices in the sample. To analyze what is the sufficient sample size for this algorithm, consider the following generalization of the birthday paradox for k-way collisions. Assume we sample elements under the uniform distribution over $[n]$. Then we obtain a k-way collision after taking $\Theta(n^{1-1/k})$ samples. Similarly, suppose we sample vertices uniformly from a graph that is composed only of n/k vertex-disjoint copies of C_k. Then, after sampling $\Theta(n^{1-1/k})$ vertices, we will hit all the vertices of at least one of the copies (with high constant probability). Conditioned on this event, if we reveal the neighborhood of all the vertices in the sample, then we obtain a C_k.

The next observation is that, in fact, we only need to hit a vertex cover of a copy of a C_k (as opposed to all its vertices). In particular we would like to hit such a cover that contains only light vertices, which we refer to as a light vertex cover. For constant α, this yields an improved dependence on k in the exponent, i.e., $O(n^{1-1/\lfloor k/2 \rfloor})$ sampled vertices suffice.

When taking into account the dependence on α (so that it is not necessarily true that $m = O(n)$) and incorporating this in the analysis, we prove that the query complexity is upper bounded by $O(m \cdot (\alpha/m)^{2/k})$ for even k and $O(m \cdot (\alpha/m)^{2/(k+1)})$ for odd k (up to a polynomial dependence on k). Since $\alpha \leq \sqrt{m}$ it follows that the above bounds are at most $O(m^{1-1/k})$ and $O(m^{1-1/(k+1)})$, respectively.

The second algorithm. Our second algorithm is designed for the case in which k is odd and $m = \Omega(\alpha^{(k+3)/2})$. In particular it is preferable when α is constant. We observe that when k is odd, for each C_k in $\mathcal{C}$, there is an edge in which both endpoints are light vertices. Therefore, if we sample edges (almost) uniformly from the graph (using a variant of the procedure Select-an-Edge), then we are likely to hit one of these edges. This additional step reduces the number of vertices we need to hit in each copy by 2, which results in improved complexity for some range of the parameters. In particular, the query complexity of this algorithm is $O(m \cdot (\alpha^2/m)^{2/(k-1)})$. Specifically, when α is a constant, the query complexity of this algorithm (which works for odd k) is $O(m^{1-2/(k-1)})$ (instead of $O(m^{1-2/(k+1)})$).

General subgraph F. Our first algorithm also works for any constant-size subgraph, F, where the upper bound on the sample size is of the form $m^{1-1/\ell(F)}$ where $\ell(F)$ depends on the structure of F, and is at most $k = |V(F)|$. The idea is that if we want to find a copy of F, it suffices to hit a light vertex cover of this copy and then query all neighbors of the sampled light vertices.

1.3 Related work

In this subsection we shortly discuss several related works, in addition to the two aforementioned works regarding testing C_3-freeness [3, 25].

Testing subgraph-freeness for fixed, constant size subgraphs in the dense-graphs model can be done using a number of queries that depends only on $1/\epsilon$ (where the dependence is a tower of height poly$(1/\epsilon)$), as

shown by Alon, Fischer, Krivelevich and Szegedy [2]. Alon [1] proved that a super polynomial dependence on $1/\epsilon$ is necessary, unless the subgraph F is bipartite. Goldreich and Ron addressed the problem in the bounded-degree model [20], and gave a simple algorithm that depends polynomially on $1/\epsilon$ and the maximum degree in the graph, and exponentially on the diameter of F.

A special case of graphs that have bounded arboricity is the family of graphs that exclude a fixed minor (a.k.a. minor-free graphs). Newman and Sohler [28] showed that for this family of graphs, in the bounded-degree model, all properties can be tested with no dependence on the size of the graph G. Moreover, it was recently shown [24, 26] that any property which is monotone and additive[14] (and in particular F-freeness where F is a connected graph) can be tested using a number of queries that is only polynomial in $1/\epsilon$ and d, where d is the degree bound (and $O(d^{\rho(\epsilon)})$ in general $(\epsilon, \rho(\epsilon))$-hyperfinite graphs[15]). For minor-free graphs with unbounded degrees, Czumaj and Sohler [10] showed that a property is testable with one sided error and a number of queries that does not depend on the size of the graph if and only if it can be reduced to testing for a finite family of finite forbidden subgraphs.[16] The correctness of their algorithm relies on the fact that the arboricity of minor-free graphs remains constant even after contractions of edges (which is not the case for general constant-arboricity graphs).

In general graphs, it was shown that k-path freeness [22] and more generally T-freeness where T is a tree of order k [17], can be tested with time and query complexity that depend only on k, assuming the edges of the graph can be accessed uniformly at random. Testing cycle-freeness (where a no instance is a graph that is far from being a forest) was studied in the bounded-degree model in [20], where a two-sided error algorithm was given whose query complexity is polynomial in $1/\epsilon$ and the degree bound. Czumaj et. al [9] showed that the complexity of this problem for one-sided error algorithms in the bounded-degree model is $\widetilde{\Theta}(\sqrt{n})$ (for constant ϵ – their algorithm has a polynomial dependence on $1/\epsilon$), and the algorithm can be adapted to the general-graphs model.

Other sublinear-time graph algorithms for counting and sampling (rather than detecting) subgraphs that give improved results when the graph G has bounded arboricity include [14, 12, 15, 13].

1.4 Organization

We start in Sections 2 with some preliminaries. In Section 3 we give the upper bound for testing C_4-freeness. Few details are deferred to Appendix A, and the lower bound for testing C_4-freeness is given in Appendix B. In Appendix C we provide the adaptation of the algorithm for testing C_4-freeness, to testing C_5-freeness (in graphs with constant arboricity). Our algorithm for testing C_6-freeness (in graphs with constant arboricity) is given in Appendix D. Our lower bound for one-sided error testing of C_k-freeness is proved in Appendix E, and the upper bound is given in Appendix F.

2 Preliminaries

Unless stated explicitly otherwise, the graphs we consider are simple, so that in particular they do not contain any parallel edges. We denote the number of vertices in the graph by n and the number of edges by m. Every vertex v in the graph has a unique id, denoted $id(v)$, and its degree is denoted by $d(v)$.

We work in what is known as the general graph model [30, 23]. In particular, under this model, the distance of a graph G to C_k-freeness, denoted $dist(G, C_k\text{-free})$, is the minimum fraction of edges that should be removed from G in order to obtain a C_k-free graph. As for the allowed queries, a neighbor query to the ith neighbor of a vertex v is denoted by $nbr(v, i)$, and to its degree by $deg(v)$. A pair query between two vertices v_1 and v_2 is denoted by $pair(v_1, v_2)$. Given query access to a graph G and a parameter ϵ, a one-sided error testing algorithm for C_k-freeness should accept G if it is C_k-free, and should reject G with probability

[14] A property is monotone if it closed under removal of edges and vertices. A property is additive if it is closed under the disjoint union of graphs.

[15] Let ρ be a function from $\mathbb{R}_+$ to $\mathbb{R}_+$. A graph $G = (V, E)$ is $(\epsilon, \rho(\epsilon))$-hyperfinite if for every $\epsilon > 0$ it is possible to remove $\epsilon|V|$ edges of the graph such that the remaining graph has connected components of size at most $\rho(\epsilon)$.

[16] They consider a model in which they can perform only random neighbor queries.

at least 2/3 if $dist(G, C_k\text{-free}) > \epsilon$. If the algorithm may also reject C_k-free graphs with probability at most 1/3, then it has two-sided error.

As noted in the introduction, we assume our algorithms for graphs whose arboricity is not promised to be constant, are given an upper bound α on the arboricity $arb(G)$ of the tested graph G, and their complexity depends on this upper bound. Alternatively, if the algorithm is provided with the number of edges, m, then it may run a procedure from [25] to obtain a value α^* that with high constant probability satisfies the following: (1) $\alpha^* \leq 2arb(G)$; (2) The number of edges between vertices whose degree is at least $\alpha^*/(c\epsilon)$ for a constant c is at most $(1 - \epsilon/c')m$ (for another, sufficiently large, constant c'). Up to polylogarithmic factors in n, the query complexity and running time of the procedure are $O(arb(G)/\epsilon^3)$ with high probability (assuming the average degree is $\Omega(1)$).

Throughout this work we assume, whenever needed, that ϵ is upper bounded by some sufficiently small constant (or else it can be set to that constant).

3 An upper bound of $\widetilde{O}(n^{1/4}\alpha)$ for testing C_4-freeness

In this subsection we prove the more general (arboricity-dependent) form of the upper bound for testing C_4-freeness which is stated as Theorem 6 in the introduction.

Recall that the assumption on α is that it is an upper bound on the arboricity $arb(G)$ - hence, while it is known that for graphs with $arb(G) > \sqrt{n}$ there exists a C_4, we cannot simply reject if we get $\alpha > n^{1/2}$ since it might be that $arb(G) < \sqrt{n}$ (and we want one-sided error). The above expression shows that in the case that $\alpha > n^{1/2}$, the $n^{1/4}\alpha$ term is replaced by $n^{3/4}$ (and the additive α term is due to the edge sampling).

The algorithm referred to in Theorem 6 is described next.

Test-C_4-freeness(n, ϵ, α)

1. Let $\theta_0 = 4\alpha/\epsilon$, $\theta_1 = c_1 \cdot \sqrt{n}/\epsilon$ (where c_1 will be determined subsequently) and $\theta_{min} = \min\{\theta_0, \theta_1\}$ (it is useful to read the algorithm while having in mind that $\theta_0 \leq \theta_1$ (i.e., $\alpha = O(\sqrt{n})$) so that $\theta_{min} = \theta_0$).
2. Repeat the following $t = \Theta(1/\epsilon)$ times:
   (a) Select an edge e by calling the procedure Select-an-Edge(α, ϵ), which is provided in Appendix A.1. If it does not return an edge, then continue to the next iteration.
   (b) Select an endpoint v of e by flipping a fair coin.
   (c) If $d(v) \leq \theta_1$, then select $s_1 = \Theta(\sqrt{d(v)/\epsilon})$ $(= O(n^{1/4}/\epsilon))$ random neighbors of v, and for each neighbor u such that $d(u) \leq \theta_{min}$ query all the neighbors of u.
   (d) Otherwise $(d(v) > \theta_1)$, perform $s_2 = \Theta(\sqrt{(n\alpha/\theta_1)\log n}/\epsilon^2)$ $(= \tilde{O}(n^{1/4}\alpha^{1/2}/\epsilon^2))$ random walks of length 2 starting from v.
   (e) If a C_4 is detected, then return it, 'Reject' and terminate.
3. Return 'Accept'.

We note that the algorithm can be unified/simplified so that it only performs random walks of length-2 where the number of walks is $\Theta(n^{1/4}\alpha/\epsilon^2)$, but then the analysis becomes slightly more complicated.

We start by stating a claim concerning the procedure Select-an-Edge – where both the procedure and the proof of the claim are deferred to Appendix A.1. We then state and prove two additional claims that will be used in the proof of Theorem 6.

Claim 1 With probability at least 2/3 the procedure Select-an-Edge returns an edge. Conditioned on it returning an edge, each edge incident to a vertex with degree at most θ_0 is returned with probability at least $1/(2m')$ and at most $1/m'$, where m' is the number of edges incident to vertices with degree at most θ_0.

Claim 2 Let v be a vertex and let $C(v,\theta_{min})$ be a set of edge-disjoint C_4's containing v such that the neighbors of v on these C_4s all have degree at most θ_{min}, where θ_{min} is as defined in the algorithm.[17] Suppose that $|C(v,\theta_{min})| \geq 1$ and let $\epsilon' = |C(v,\theta_{min})|/d(v)$. If we select $s = 16\sqrt{d(v)/\epsilon'}$ random neighbors of v, and for each selected neighbor u such that $d(u) \leq \theta_{min}$ we query all the neighbors of u, then the probability that we obtain a C_4 is at least 9/10.

Proof: Let $E'(v)$ denote the set of edges incident to v that participate in the set $C(v,\theta_{min})$. By the premise of the claim, $|E'(v)|/d(v) = 2|C(v,\theta_{min})|/d(v) = 2\epsilon'$. Let s' be the number of neighbors of v that are incident to edges in $E'(v)$ among the s selected random neighbors of v. It holds that $\mathbb{E}[s'] = 2\epsilon' \cdot s$, and by the multiplicative Chernoff bound, $s' \geq \epsilon' \cdot s$ with probability at least $1 - e^{-\epsilon' \cdot s/4}$. We first show that this probability is at least 19/20, and then condition on this event. By the setting of $s = 16\sqrt{d(v)/\epsilon'}$, it holds that $\epsilon' \cdot s = 16\sqrt{\epsilon' \cdot d(v)}$, and by the setting of $\epsilon' = |C(v,\theta)|/d(v)$, we get $\epsilon' \cdot s \geq 16\sqrt{|C(v,\theta_{min})|} \geq 16$. Therefore, with probability at least 19/20, $s' > \epsilon' \cdot s = 16\sqrt{\epsilon' \cdot d(v)}$. We condition on this event and consider only those s' selected neighbors of v that are endpoints of $E'(v)$.

For each 4-cycle $\rho \in C(v,\theta_{min})$, let $u_1(\rho)$ and $u_2(\rho)$ be the two neighbors of v on this C_4 (so that they are endpoints of edges in $E'(v)$). Since the C_4s in $C(v,\theta_{min})$ are edge-disjoint, these vertices are distinct. Observe that the s' selected neighbors of v are uniformly distributed in $\bigcup_{\rho \in C(v,\theta_{min})}\{u_1(\rho), u_2(\rho)\}$, and that $s' \geq 16 \cdot \sqrt{|C(v,\theta_{min})|}$. Hence, by the "birthday paradox", with high constant probability, the sample of neighbors of v contains two vertices, $u_1(\rho)$, and $u_2(\rho)$ for some $\rho \in C(v,\theta_{min})$. Conditioned on this event, once the (at most θ_{min}) neighbors of $u_1(\rho)$ and $u_2(\rho)$ are queried, the four-cycle ρ is observed. □

Claim 3 Let G be a graph over n vertices and m edges, and let $\theta_1, \epsilon', \epsilon''$ be parameters. Suppose that G contains a bipartite subgraph $G' = (L, R, E(G'))$ such that every vertex in R has degree at least θ_1 in G. Let v be a vertex in R such that v has at least $\epsilon' \cdot d(v)$ neighbors in L where each of these neighbors, u, has at least $\epsilon'' \cdot \max\{d(u), \frac{m}{n}\}$ neighbors in R. If $\theta_1 \geq 2\sqrt{n/(\epsilon' \cdot \epsilon'')}$ and we take $s_2 \geq \frac{32}{\epsilon' \cdot \epsilon''} \cdot \sqrt{2 \log n \cdot |R|}$ random walks of length 2 from v for a sufficiently large constant c', then with probability at least 9/10 we shall detect a C_4 in G.

Proof: For a pair of vertices v and $v' \neq v$ in R, let $\ell_2(v,v')$ be the number of length-2 paths between v and v', and let $\ell_2(v,R) = \sum_{v' \in R} \ell_2(v,v')$. Consider taking two random length-2 walks from v, and let $\mathcal{E}_1$ be the event that both of them end at vertices in R. Let $\mathcal{E}_2$ be the event that these two paths are distinct and end at the same vertex. Then for each single vertex $v' \in R$, conditioned on $\mathcal{E}_1$, the probability that the two walks end at v' is exactly $\frac{\ell_2(v,v')}{\ell_2(v,R)} \cdot \frac{\ell_2(v,v')-1}{\ell_2(v,R)}$. Therefore,

$$\Pr[\mathcal{E}_2 \,|\, \mathcal{E}_1] = \sum_{v' \in R} \frac{\ell_2(v,v')}{\ell_2(v,R)} \cdot \frac{\ell_2(v,v')-1}{\ell_2(v,R)} = \frac{1}{(\ell_2(v,R))^2} \cdot \sum_{v' \in R} (\ell_2(v,v'))^2 - \frac{1}{\ell_2(v,R)} \,. \tag{1}$$

We would like to lower bound the above probability. For the first term on the right-hand-side, by applying the Cauchy-Schwartz inequality we get that

$$\frac{1}{(\ell_2(v,R))^2} \cdot \sum_{v' \in R} (\ell_2(v,v'))^2 \geq \frac{1}{(\ell_2(v,R))^2} \cdot |R| \cdot \left(\frac{\ell_2(v,R))}{|R|}\right)^2 = \frac{1}{|R|} \,. \tag{2}$$

By combining Equations (1) and (2) we get that $\Pr[\mathcal{E}_2 \,|\, \mathcal{E}_1] \geq \frac{1}{|R|} - \frac{1}{\ell_2(v,R)}$. Since each vertex in R has degree at least θ_1, we have that $|R| \leq \frac{2m}{\theta_1}$. By the premise of the claim regarding v and its neighbors, v has $\epsilon' d(v) \geq \epsilon' \cdot \theta_1$ neighbors in L, and each of them has at least $\epsilon'' \cdot (m/n)$ neighbors in R. Therefore,

$$\ell_2(v,R) \geq \epsilon' \cdot \theta_1 \cdot \epsilon'' \cdot \frac{m}{n} \geq \frac{\epsilon' \cdot \epsilon'' \cdot \theta_1^2 \cdot |R|}{2n} \geq 2|R|, \tag{3}$$

[17] Actually, we do not rely on the setting of θ_{min}, so this claim holds for any threshold value.

where the last inequality is by the premise $\theta_1 \geq 2\sqrt{n/(\epsilon' \cdot \epsilon'')}$. Therefore, $\Pr[\mathcal{E}_2 \,|\, \mathcal{E}_1] \geq \frac{1}{2|R|}$. So far we have shown that when taking two distinct random walks from v, and conditioned on them both ending at R (the event $\mathcal{E}_1$), the two paths collide on the end vertex (and hence result in a C_4) with probability at least $1/2|R|$. We shall now prove, that when taking s length-2 random walks from v, sufficiently many of them indeed end at R, and that with high probability, at least two of them collide, resulting in a C_4.

Consider first the event $\mathcal{E}_1$. By the premise of the claim, v has at least $\epsilon' \cdot d(v)$ neighbors in L, and each u of them has at least $\epsilon'' \max\{d(u), m/n\} \geq \epsilon'' d(u)$ neighbors in R. Therefore, the probability that a single random walk from v ends at R is at least $\epsilon' \cdot \epsilon''$. Hence, if we take $s \geq \frac{32}{\epsilon' \cdot \epsilon''}\sqrt{2 \log n \cdot |R|}$ length-2 random walks from v, and let s' denote the number of walks that end at a vertex in R, we have that $\mathbb{E}[s'] = 32 \cdot \sqrt{2 \log n \cdot |R|}$, and that with probability at least $9/10$, we have $s' \geq 16 \cdot \sqrt{2 \log n \cdot |R|}$. We henceforth condition on this event.

Let $\chi'_{i,j}$ denote the event that the ith and jth random walks among the ones that end at R collide on the ending vertex (and thus result in a C_4). By the above discussion, we have that for a specific pair $i \neq j$, $\Pr[\chi'_{i,j} = 1] \geq 1/2|R|$. We now lower bound the probability that at least one pair of random walks from the s' that end in R detects a C_4, i.e. lower bound $\sum_{i,j \in [s']} \chi_{i,j}$, using Corollary 16. For that end we also need to upper bound the variance of the sum.

Partition the vertices in R according to $\ell_2(v, v')$, where $R_x(v) = \{v' : 2^{x-1} < \ell_2(v, v') \leq 2^x\}$ for $x = 0, \ldots \log L \leq \log n$. Since $\sum_{v' \in R} \frac{\ell_2(v,v')}{\ell_2(v,R)} \cdot \frac{\ell_2(v,v')-1}{\ell_2(v,R)} > \frac{1}{2|R|}$, there exists at least one setting of x for which $\sum_{v' \in R_x} \frac{\ell_2(v,v')}{\ell_2(v,R)} \cdot \frac{\ell_2(v,v')-1}{\ell_2(v,R)} \geq \frac{1}{2|R| \log n}$. We denote this setting by x^* and observe that $x^* > 0$ (since for every $v' \in R_0$, $\ell_2(v, v') - 1 = 0$).

For every $i, j \in [s']$, $i < j$, we define a Bernoulli random variable $\chi_{i,j}$ that is 1 if and only if the ith and the jth random walks from v (among the s' considered) end at the same $v' \in R_{x^*}$ and pass through a different vertex in L. We next show that we can apply Corollary 16 (with s in that claim set to s') to get an upper bound on the probability that $\sum_{i,j \in [s'], i<j} \chi_{i,j} = 0$ (which is an upper bound on the probability that we do not detect a C_4).

By the definition of the random variables, for every $i_1 \neq i_2, j_1 \neq j_2$, it holds that $\chi_{i_1,j_1}, \chi_{i_2,j_2}$ are independent, so that the first condition in Corollary 16 is satisfied. Next, for any pair $i, j \in [s']$, $i < j$ we have that

$$\mu = \Pr[\chi_{i,j} = 1] = \sum_{v' \in R_{x^*}} \frac{\ell_2(v, v')}{\ell_2(v, R)} \cdot \frac{\ell_2(v, v') - 1}{\ell_2(v, R)} \geq \frac{1}{2|R| \log n} \,. \tag{4}$$

Therefore, we have that $s' \geq 16 \cdot \sqrt{2|R| \log n} = 16/\sqrt{\mu}$, and so the third condition in Corollary 16 is satisfied (for $c_2 = 16$, where s' serves as the parameter s in the corollary).

It remains to verify that the second condition holds. For any four indices $i_1, j_1, i_2, j_2 \in [s']$, $i_1 < j_1$, $i_2 < j_2$ such that exactly two of the four indices are the same,

$$\Pr[\chi_{i_1,j_1} = \chi_{i_2,j_2} = 1] = \sum_{v' \in R_{x^*}} \frac{\ell_2(v, v')}{\ell_2(v, R)} \cdot \left(\frac{\ell_2(v, v') - 1}{\ell_2(v, R)}\right)^2 \leq \mu \cdot \frac{2^{x^*} - 1}{\ell_2(v, R)} \,. \tag{5}$$

Since by Equation (4), $\mu = \sum_{v' \in R_{x^*}} \frac{\ell_2(v,v')}{\ell_2(v,R)} \cdot \frac{\ell_2(v,v')-1}{\ell_2(v,R)} \geq \frac{2^{2(x^*-1)}}{2(\ell_2(v,R))^2}$ (as $\ell_2(v, v') \geq 2^{x^*-1}$ for every $v' \in R_{x^*}$ and $\ell_2(v, v') - 1 \geq \ell_2(v, v')/2$), we get that $\Pr[\chi_{i_1,j_1} = \chi_{i_2,j_2} = 1] < \sqrt{2} \cdot \mu^{3/2}$, and so the second condition in Corollary 16 holds as well (for $c_1 = \sqrt{2}$). Thus, the current claim follows. □

We are now ready to prove Theorem 6.

Proof of Theorem 6: Since the algorithm only rejects a graph G if it detects a C_4, it will always accept graphs that are C_4-free. Hence, we focus on the case that G is ϵ-far from being C_4-free.

Recall that $\theta_0 = 4\alpha/\epsilon$ and let $E_{>\theta_0}$ be the subset of edges in G where both endpoints have degree greater than θ_0. Since the arboricity of G is at most α, there are at most $2m/\theta_0$ vertices with degree greater than θ_0, so that $|E_{>\theta_0}| \leq (2m/\theta_0) \cdot \alpha = \epsilon m/2$ edges.

Since G is ϵ-far from C_4-free, if we remove all edges in $E_{>\theta_0}$, then we get a graph that is at least $(\epsilon/2)$-far from C_4-free. It follows that there exists a set of edge-disjoint C_4s, denoted $\mathcal{C}$, such that no C_4 in $\mathcal{C}$ contains an edge in $E_{>\theta_0}$, and $|\mathcal{C}| \geq \epsilon m/8$.

We next partition $\mathcal{C}$ into two disjoint subsets: $\mathcal{C}_1$ contains those C_4s that have at most one vertex with degree at least θ_1 in them, and $\mathcal{C}_2$ contains those that have at least two such vertices (where in the case $\theta_0 \leq \theta_1$ there will be exactly two). Since $\mathcal{C} = \mathcal{C}_1 \cup \mathcal{C}_2$, either $|\mathcal{C}_1| \geq \epsilon m/16$ or $|\mathcal{C}_2| \geq \epsilon m/16$ (possibly both).

The case $|\mathcal{C}_1| \geq \epsilon$m$/16$. Consider first the case that $|\mathcal{C}_1| \geq \epsilon m/16$. In order to analyze this case, we apply a process of "coloring" vertices and edges. Initially, all vertices and edges that participate in C_4s that belong to $\mathcal{C}_1$ are colored green, and all other vertices and edges are colored red. We next apply the following iterative process. As long as there is a green vertex v whose number of incident green edges is less than $\epsilon d(v)/64$, color v and its green incident edges by red. Observe that the total number of edges colored red by this process is at most $\epsilon m/32$. Furthermore, at the end of this process, every green vertex v has at least $\epsilon d(v)/64$ incident green edges (and if a vertex is red, then all its incident edges are red). Let $\mathcal{C}'_1$ be the subset of $\mathcal{C}_1$ that consists of those C_4s in $\mathcal{C}_1$ whose edges all remain green after the process (and hence they are green), so that $|\mathcal{C}'_1| \geq \epsilon m/32$.

By the definition of $\mathcal{C}_1$, and hence also $\mathcal{C}'_1$, in each C_4 in $\mathcal{C}'_1$ there is at most one vertex with degree greater than θ_1, and no edges such that both endpoints have degree greater than θ_0. Assume without loss of generality that for each four-cycle $\rho \in \mathcal{C}'_1$, where $\rho = (v_0(\rho), v_1(\rho), v_2(\rho), v_3(\rho))$, $v_2(\rho)$ is the highest degree vertex (where $d(v_2(\rho))$ could be any value between 1 to n). Let $V_0(\mathcal{C}'_1) = \bigcup_{\rho \in \mathcal{C}'_1} \{v_0(\rho)\}$ denote this set of vertices (i.e., the ones that are across from the highest degree vertex in a (green) four-cycle in $\mathcal{C}'_1$).

Observation: For every $\rho \in \mathcal{C}'_1$, (1) $d(v_0(\rho)) \leq \theta_1$, and (2) $v_1(\rho)$ and $v_3(\rho)$ are of degree at most $\theta_{min} = \min\{\theta_0, \theta_1\}$.

To verify this observation, note that by the definition of $\mathcal{C}'_1$, for every $\rho \in \mathcal{C}'_1$, there is at most one vertex with degree greater than θ_1, and since $v_2(\rho)$ is the highest degree vertex in ρ, it follows that all three other vertices in ρ are of degree at most θ_1.

We now show that $d(v_1(\rho)) \leq \theta_0$, and the proof for $v_3(\rho)$ is identical. If $d(v_2(\rho)) > \theta_0$, then it must be the case that $d(v_1(\rho)) < \theta_0$, as otherwise both have degree greater than θ_0 and so they cannot be connected, which is a contradiction to them both being incident on the four-cycle ρ. If $d(v_2(\rho)) \leq \theta_0$, then since $v_2(\rho)$ is the highest degree vertex in ρ, $d(v_1(\rho)) \leq d(v_2(\rho)) \leq \theta_0$.

Therefore, for every $v \in V_0(\mathcal{C}'_1)$, it has at least $\epsilon d(v)/64$ neighbors u such that (v, u) is green and $d(u) \leq \theta_{min}$. Hence, overall in the graph, the set of vertices $V_0(\mathcal{C}'_1)$ has at least $\epsilon m/32$ green edges that are incident to it and their second endpoint is of degree at most $\theta_{min} \leq \theta_0$. It follows that conditioned on an edge being returned by procedure Select-an-Edge, by Claim 1, it returns an edge incident to a vertex $v \in V_0(\mathcal{C}'_1)$ with probability at least $(\epsilon m/32)/2m' > \epsilon/128$ (since $m' > \frac{1}{2}m$). So the probability that in some iteration of Test-C_4-freeness a vertex $v_0 \in V_0(\mathcal{C}'_1)$ is selected, is at least $1 - (1 - \frac{\epsilon}{128})^t > 9/10$ (recall that $t = \Theta(1/\epsilon)$ so that it suffices to set $t = 500/\epsilon$).

Conditioning on this event, we apply Claim 2. Specifically:

- $\theta_0 = 4\alpha/\epsilon$ (as defined in Step 1 in Algorithm Test-C_4-freeness);
- $C(v_0, \theta_{min})$ is the set of C_4s in $\mathcal{C}'_1$ that are incident to v_0;
- $\epsilon' = |C(v_0, \theta_{min})|/d(v) \geq \epsilon/128$ (since v_0 has at least $\epsilon d(v)/64$ incident green edges, and they can be partitioned into pairs such that each pair belongs to exactly one C_4 in $C(v_0, \theta_{min})$);
- $d(v_0) \leq \theta_1$ (by the above observation);

In order to apply the claim, we must ensure that $s > 16\sqrt{d(v_0)/\epsilon'}$. By the above, it is sufficient to set s_1 in Step 2c, to be $s_1 = 512\sqrt{d(v_0)/\epsilon}$.

Hence, by Claim 2, if Step 2c is applied to v_0, then a C_4 is observed with probability at least 9/10.

The analysis for the case that $|\mathcal{C}_2| \geq \epsilon m/16$ is similar, and due to space constraints, it is deferred to Appendix A.2.

We next turn to analyze the query complexity. By the settings of θ_0, θ_1, t, s_1 and s_2 in the algorithm, the query complexity of the algorithm is upper bounded as follows.

$$O\left(\frac{1}{\epsilon}\cdot\left(\frac{\alpha}{\epsilon}+\max\{s_1,s_2\}\right)\right)=O\left(\frac{1}{\epsilon}\left(\frac{\alpha}{\epsilon}+\max\left\{\sqrt{\frac{\theta_1}{\epsilon}}\cdot\theta_{min},\frac{1}{\epsilon^2}\cdot\sqrt{\frac{n\alpha}{\theta_1}\cdot\log n}\right\}\right)\right) \tag{6}$$

For the case that $\alpha \leq (c_1/4)\sqrt{n}$, we have that $\theta_{min}=\theta_0=\Theta(\alpha/\epsilon)$ and that $\theta_1=\Theta(\sqrt{n}/\epsilon)$, and so we get a complexity of

$$O\left(\epsilon^{-3}\cdot n^{1/4}\alpha^{1/2}\cdot\max\{\alpha^{1/2},\log^{1/2}n\}\right)=O(\epsilon^{-3}\cdot n^{1/4}\alpha)\ , \tag{7}$$

where the last inequality is for $\alpha > \log n$, and otherwise the complexity is $O(\epsilon^{-3}\cdot n^{1/4}\alpha^{1/2}\log^{1/2}n)$.

For the case that $\alpha > (c_1/4)\sqrt{n}$, we have that $\theta_{min}=\theta_1=\Theta(\sqrt{n}/\epsilon)$. Therefore, the complexity is

$$O(\epsilon^{-3}\cdot(\alpha+n^{3/4}))\ . \tag{8}$$

Thus, the proof is complete. □

References

[1] Noga Alon. Testing subgraphs in large graphs. Random Struct. Algorithms, 21(3-4):359–370, 2002.

[2] Noga Alon, Eldar Fischer, Michael Krivelevich, and Mario Szegedy. Efficient testing of large graphs. Combinatorica, 20(4):451–476, 2000.

[3] Noga Alon, Tali Kaufman, Michael Krivelevich, and Dana Ron. Testing triangle-freeness in general graphs. SIAM Journal on Discrete Mathematics, 22(2):786–819, 2008.

[4] Noga Alon, Raphael Yuster, and Uri Zwick. Color coding. Journal of the ACM, 42(4):844—856, 1995.

[5] Uri Alon. Network motifs: theory and experimental approaches. Nature Reviews Genetics, 8(6):450–461, 2007.

[6] Jonathan W. Berry, Bruce Hendrickson, Randall A. LaViolette, and Cynthia A. Phillips. Tolerating the Community Detection Resolution Limit with Edge Weighting. Physical Review E, 83(5):056119, May 2011.

[7] Ronald S. Burt. Structural holes and good ideas. American journal of sociology, 110(2):349–399, 2004.

[8] James S. Coleman. Social capital in the creation of human capital. American Journal of Sociology, 94:S95–S120, 1988.

[9] Artur Czumaj, Oded Goldreich, Dana Ron, C. Seshadhri, Asaf Shapira, and Christian Sohler. Finding cycles and trees in sublinear time. Random Structures and Algorithms, 45(2):139–184, 2014.

[10] Artur Czumaj and Christian Sohler. A characterization of graph properties testable for general planar graphs with one-sided error (it's all about forbidden subgraphs). In 60th IEEE Annual Symposium on Foundations of Computer Science, FOCS, pages 1525–1548, 2019.

[11] Jean-Pierre Eckmann and Elisha Moses. Curvature of co-links uncovers hidden thematic layers in the World Wide Web. Proceedings of the National Academy of Sciences, 99(9):5825–5829, 2002.

[12] Talya Eden, Dana Ron, and Will Rosenbaum. The arboricity captures the complexity of sampling edges. In 46th International Colloquium on Automata, Languages, and Programming, ICALP, pages 52:1–52:14, 2019.

[13] Talya Eden, Dana Ron, and Will Rosenbaum. Almost optimal bounds for sublinear-time sampling of k-cliques in bounded arboricity graphs. In 49th International Colloquium on Automata, Languages, and Programming, ICALP, pages 56:1–56:19, 2022.

[14] Talya Eden, Dana Ron, and C. Seshadhri. Sublinear time estimation of degree distribution moments: The arboricity connection. SIAM J. Discret. Math., 33(4):2267–2285, 2019.

[15] Talya Eden, Dana Ron, and C. Seshadhri. Faster sublinear approximation of the number of k-cliques in low-arboricity graphs. In Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA, pages 1467–1478, 2020.

[16] Paul Erdős. Extremal problems in graph theory. In In Proc. Symp. Theory of Graphs and its Applications, pages 29–36, 1963.

[17] Guy Even, Orr Fischer, Pierre Fraigniaud, Tzlil Gonen, Reut Levi, Moti Medina, Pedro Montealegre, Dennis Olivetti, Rotem Oshman, Ivan Rapaport, and Ioan Todinca. Three notes on distributed property testing. In 31st International Symposium on Distributed Computing, DISC, pages 15:1–15:30, 2017.

[18] Brooke Foucault Welles, Anne Van Devender, and Noshir Contractor. Is a friend a friend?: Investigating the structure of friendship networks in virtual worlds. In CHI Extended Abstracts on Human Factors in Computing Systems, pages 4027–4032, 2010.

[19] Oded Goldreich. Open problems in property testing of graphs. In Electron. Colloquium Comput. Complex., volume 28, page 88, 2021.

[20] Oded Goldreich and Dana Ron. Property testing in bounded degree graphs. Algorithmica, 32(2):302–343, 2002.

[21] Paul W. Holland and Samuel Leinhardt. A method for detecting structure in sociometric data. American Journal of Sociology, 76:492–513, 1970.

[22] Kazuo Iwama and Yuichi Yoshida. Parameterized testability. ACM Trans. Comput. Theory, 9(4):16:1–16:16, 2018.

[23] Tali Kaufman, Michael Krivelevich, and Dana Ron. Tight bounds for testing bipartiteness in general graphs. SIAM Journal on Computing, 33(6):1441–1483, 2004.

[24] Akash Kumar, C. Seshadhri, and Andrew Stolman. Random walks and forbidden minors III: poly($d\varepsilon^{-1}$)–time partition oracles for minor-free graph classes. In 62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS, pages 257–268, 2021.

[25] Reut Levi. Testing triangle freeness in the general model in graphs with arboricity $o(\sqrt{n})$. In 48th International Colloquium on Automata, Languages, and Programming, ICALP, volume 198, pages 93:1–93:13, 2021.

[26] Reut Levi and Nadav Shoshan. Testing Hamiltonicity (and other problems) in minor-free graphs. In Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM, pages 61:1–61:23, 2021.

[27] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: simple building blocks of complex networks. Science, 298(5594):824–827, 2002.

[28] Ilan Newman and Christian Sohler. Every property of hyperfinite graphs is testable. SIAM Journal on Computing, 42(3):1095–1112, 2013.

[29] Matthew O., Tomas Rodriguez-Barraquer, and Xu Tan. Social capital and social quilts: Network patterns of favor exchange. American Economic Review, 102(5):1857?1897, 2012.

[30] Michal Parnas and Dana Ron. Testing the diameter of graphs. Random Struct. Algorithms, 20(2):165–183, 2002.

[31] Alejandro Portes. Social capital: Its origins and applications in modern sociology. In Eric L. Lesser, editor, Knowledge and Social Capital, pages 43 – 67. Butterworth-Heinemann, Boston, 2000.

[32] C. Seshadhri, Tamara G. Kolda, and Ali Pinar. Community structure and scale-free collections of Erdös-Rényi graphs. Physical Review E, 85(5):056109, May 2012.

A Missing details for the testing algorithm for C_4

A.1 The procedure **Select-an-Edge** and Proof of Claim 1

Select-an-edge(ϵ, α)

1. Repeat the following $\Theta(\alpha/\epsilon)$ times:
   (a) Select a vertex u uniformly at random.
   (b) If $d(u) \leq \theta_0$ for $\theta_0 = 4\alpha/\epsilon$, then with probability $d(u)/\theta_0$ select an edge incident to u uniformly at random and return it.
2. If no edge was selected, then return 'Fail'.

Proof of Claim 1: Let L denote the set of vertices with degree at most θ_0. Consider any single iteration of the algorithm. Since $m \leq n\alpha$, the fraction of vertices not in L is at most ϵ. Hence, the probability of selecting a vertex in L is at least $1 - \epsilon > 1/2$. Conditioned on this event, the probability that an edge is returned is at least $1/\theta_0$. Since the procedure performs $\Theta(\theta_0)$ iterations, for an appropriate constant in the Θ notation, an edge is returned with probability at least $2/3$.

Turning to the second part of the claim, let E_1 denote the set of edges that have a single endpoint in L and let E_2 denote the set of edges that have both endpoints in L. Conditioned on an edge being selected, an edge that has a single endpoint in L is selected with probability $\frac{1}{|E_1|+2|E_2|} \geq \frac{1}{2m'}$ and an edge with both endpoints in L is selected with probability $\frac{2}{|E_1|+2|E_2|} \leq \frac{1}{m'}$. □

A.2 Missing details in the proof of Theorem 6

The case $|\mathcal{C}_2| \geq \epsilon m/16$. We now turn to the case that $|\mathcal{C}_2| \geq \epsilon m/16$, where recall that $\mathcal{C}_2$ is the set of green C_4s with at least two vertices with degree greater than θ_1. We again consider two sub-cases, depending on whether $\theta_0 \leq \theta_1$ or not. We start with the former case $\theta_0 \leq \theta_1$ (i.e., $\alpha \leq (c_1/4)\sqrt{n}$).

The sub-case $\alpha \leq (c_1/4)\sqrt{n}$. Recall that by the definition of $\mathcal{C}_2$, for every 4-cycle $\rho \in \mathcal{C}_2$, it has no edges with two endpoints greater than θ_0. Since $\theta_0 \leq \theta_1$, this also implies that ρ has exactly two vertices with degree greater than θ_1. Hence for each $\rho \in \mathcal{C}_2$, there are two vertices v, v' with degree greater than θ_1 that do not neighbor one another on the C_4, and there are two vertices u, u' with degree at most θ_0 that each neighbors both v and v'. Thus the vertices and edges that participate in C_4s belonging to $\mathcal{C}_2$ induce a bipartite graph between vertices with degree greater than θ_1 and vertices with degree at most θ_0. Here too we run a coloring process, where initially all vertices and edges that participate in C_4s contained in $\mathcal{C}_2$ are colored green, but the process is slightly modified as explained next. Let $\bar{d} = m/n$. In addition to the (re)coloring rule (green to red) defined above, we also color red a green vertex u and its incident green edges, if the number of these edges is less than $\epsilon\bar{d}/128$. This process removes at most $\epsilon m/32 + \epsilon m/64$ edges, and therefore at the end of the process we are still left with $\epsilon m/64$ many C_4's.

Let $\mathcal{C}_2'$ be the subset of $\mathcal{C}_2$ that consists of those C_4s in $\mathcal{C}_2$ whose edges all remain green, and observe that by the above process, each vertex that belongs to a C_4 in $\mathcal{C}_2'$ has at least $\max\{\epsilon\bar{d}/128, \epsilon d(v)/64\} \geq \epsilon'' \cdot \max\{\bar{d}, \epsilon d(v)\}$ many incident green edges (for $\epsilon'' = \epsilon/128$). We have that $|\mathcal{C}_2'| \geq \epsilon m/32$, and if we let R be the subset of vertices with degree at least θ_1 that are colored green, and L be the subset of

vertices with degree at most θ_0 that are colored green, then the premises of Claim 3 hold for every $v \in R$. Specifically, for (1) $\epsilon' = \epsilon/16$ and $\epsilon'' = \epsilon/128$, (2) $\theta_1 = c_1\sqrt{n}/\epsilon$ (where here $c_1 = 100$ is sufficient so that indeed $\theta_1 > 2\sqrt{n/(\epsilon' \cdot \epsilon'')}$ as required) (3) $|R| \leq 2m/\theta_1 \leq 2n\alpha/\theta_1$, so that for an appropriate setting of the constant in the $\Theta(\cdot)$ notation for $s_2 = \Theta(\sqrt{(n\alpha/\theta_1)\log n}/\epsilon^2)$ in Step 2c of the algorithm, we have that $s_2 \geq \frac{32}{\epsilon'\epsilon''}\sqrt{2|R|\log n}$, as required.

The argument for why the algorithm will select a vertex $v \in R$ with probability at least 9/10 is as in the case that $|\mathcal{C}_1| \geq \epsilon m/16$. Conditioned on this event, by Claim 3 and the setting of θ_1, when Step 2d is applied to v, a C_4 is observed with probability at least 9/10.

The sub-case $\alpha > (c_1/4)\sqrt{n}$. Recall that the set $\mathcal{C}_2$ consists of all C_4s that have at least two vertices with degree at least θ_1, and no two incident vertices where both have degree greater than θ_0. In the case where $\alpha > (c_1/4)\sqrt{n}$, so that $\theta_0 > \theta_1$, it is no longer true that all C_4s in $\mathcal{C}_2$ have exactly two vertices with degree greater than θ_1. Indeed, as $\theta_1 < \theta_0$, it can occur that the four-cycles in $\mathcal{C}_2$ have edges with both endpoints of degrees greater than θ_1. Therefore, we need a different argument in order to obtain the bipartite graph (L, R) for which we would apply Claim 3.

Consider further partitioning $\mathcal{C}_2$ into two subsets: $\mathcal{C}_{2,2}$ consists of those C_4s in $\mathcal{C}_2$ that contain exactly two vertices with degree greater than θ_0, and $\mathcal{C}_{2,1}$ of those that contain at most one vertex with degree greater than θ_0.

If $|\mathcal{C}_{2,2}| \geq |\mathcal{C}_2|/2$, then the argument proceeds essentially the same as for the sub-case $\alpha \leq (c_1/4)\sqrt{n}$ with $\mathcal{C}_2$ replaced by $\mathcal{C}_{2,2}$. Here we use the fact that $\theta_0 > \theta_1$, so that after running the "coloring" process on the vertices and edges of C_4s in $\mathcal{C}_{2,2}$, if we let R be the subset of vertices with degree greater than θ_0 that are colored green, and L be the subset of vertices with degree at most θ_0 that are colored green, then the premises of Claim 3 hold for every $v \in R$.

If $|\mathcal{C}_{2,1}| > |\mathcal{C}_2|/2$, then we do the following. First, we put in a subset $\widetilde{L}$ all vertices with degree at most θ_1, and in a subset $\widetilde{R}$ all vertices with degree greater than θ_0. As for the vertices with degree between θ_1 and θ_0, consider randomly partitioning them into the two sides. Observe that for each fixed C_4 in $\mathcal{C}_{2,1}$, the probability that all its edges cross between $\widetilde{L}$ and $\widetilde{R}$, which we refer to as "surviving", is at least 1/8, and so the expected fraction that survive is at least 1/8. Therefore, there exists a partition $(\widetilde{L}^*, \widetilde{R}^*)$ for which at least this fraction survives. We denote the set of surviving C_4s with respect to $(\widetilde{L}^*, \widetilde{R}^*)$ by $\widetilde{\mathcal{C}}_{2,1}$.

The important thing to observe is that all vertices in $\widetilde{L}^*$ have degree at most θ_0, all vertices in $\widetilde{R}^*$ have degree at least θ_1, and all edges participating in C_4s that belong to $\widetilde{\mathcal{C}}_{2,1}$ have two vertices in each side, with the edges crossing between them. The argument continues essentially as in the subcase that $\alpha \leq (c_1/4)\sqrt{n}$, where $\mathcal{C}_2$ is replaced with $\widetilde{\mathcal{C}}_{2,1}$.

B A lower bound of $\Omega(n^{1/4}\alpha^{1/2})$ for testing C_4-freeness

B.1 A two-sided error lower bound for constant-arboricity graphs

We start by proving the lower bound for constant-arboricity graphs, as stated in Theorem 1 in the introduction.

Theorem 8 Testing C_4-freeness in constant-arboricity graphs over n vertices requires $\Omega(n^{1/4})$ queries for constant ϵ.

We note that Theorem 8 holds for two-sided error algorithms and not only one-sided error ones.

Proof: For any (sufficiently large) n we shall define two bipartite graphs, G_0 and G_1, where both have constant arboricity. The graph G_0 is C_4-free and the graph G_1 is $\Omega(1)$-far from being C_4 free. Based on each G_b, $b \in \{0, 1\}$ we have a distribution, $\mathcal{D}_b$, whose support consists of graphs isomorphic to G_b (that differ only in vertex and edge labelings), and each distribution is uniform over its support. We shall show that for a sufficiently large constant c, an algorithm that performs less than $n^{1/4}/c$ queries, cannot distinguish with success probability at least 2/3 between a graph selected according to $\mathcal{D}_0$ and a graph selected according to $\mathcal{D}_2$. The lower bound stated in the theorem follows.

We start by defining G_0. The vertices in G_0 are partitioned into three sets: X_0, Y_0, Z_0, whose sizes are x_0, y_0 and z_0, respectively. These sizes satisfy the following. First, $x_0 = y_0(y_0-1)/2$. We set y_0 to be the largest odd integer satisfying $x_0 + y_0 \leq n$ and $z_0 = n - (x_0 + y_0)$. For each pair of vertices in Y_0 there is a unique vertex in X_0 that neighbors exactly these two vertices. The vertices in Z_0 have no incident edges. By construction, G_0 is C_4-free, and we have $y_0 = \Theta(n^{1/2})$, $z_0 = O(n^{1/2})$, and $x_0 = \Theta(n)$.

Turning to G_1, its vertices are also partitioned into three sets: X_1, Y_1, Z_1, of sizes x_1, y_1 and z_1, respectively, where X_1 is further partitioned into two equal-size sets X_1^1 and X_1^2, and Y_1 is partitioned into two equal-size sets Y_1^1 and Y_1^2 (so that x_1 and y_1 are both even). Here $x_1/2 = (y_1/2)^2$ so that $x_1 = y_1^2/2$, and we set $y_1 = y_0 - 1$ and $z_1 = n - (x_1 + y_1)$. For each pair of vertices, $u^1 \in Y_1^1$ and $u^2 \in Y_1^2$ there is one vertex in X_1^1 that neighbors exactly these two vertices, and one vertex in X_1^2 that neighbors them. The vertices in Z_1 have no incident edges. By construction, since there are $x_1/2 = \Theta(n)$ edge-disjoint C_4s in G_1, it is $\Omega(1)$-far from being C_4-free.

Comparing the two graphs we observe that the vertices in Z_1 have degree 0, which is the same as those in Z_0, The vertices in X_1 have degree 2, which is the same as those in X_0, and the vertices in Y_1 have degree $2 \cdot (y_1/2) = y_1 = y_0 - 1$, which is the same as those in Y_0. Comparing the sizes of the sets of vertices, since $y_1 = y_0 - 1$, we get that

$$x_1 = \frac{y_1^2}{2} = \frac{(y_0-1)^2}{2} = \frac{y_0(y_0-1)}{2} - \frac{y_0-1}{2} = x_0 - \Theta(n^{1/2}) \ ,$$

and

$$z_1 = n - (x_1 + y_1) = n - (x_0 + (1/2)(y_0 - 1)) = z_0 + y_0/2 + 1/2 = O(n^{1/2}) \ .$$

We next define two processes, $\mathcal{P}_0$ and $\mathcal{P}_1$, where $\mathcal{P}_b$, for $b \in \{0,1\}$ constructs "on the fly" a graph selected according to $\mathcal{D}_b$ while answering the queries of a testing algorithm. We shall show that unless one of several events occurs, the distributions induced on the sequences of queries and answers are identical. Let q^t denote the tth query of the algorithm and a^t the answer it gets, so that both are random variables. The query q^t may depend on the query-answer history $(q^1, a^1), \ldots, (q^{t-1}, a^{t-1})$ and possibly randomness of the algorithm. Both processes maintain a Knowledge graph, which contains the vertices and edges that appear in these questions and answers, where $K^t = (V^t, E^t)$ is the knowledge graph after the tth query is answered. Let X^t, Y^t and Z^t denote the subsets of vertices with degree 2, $y_0 - 1$ and 0, respectively. (In principle, the knowledge graph should also include information about the degrees of vertices and pairs of vertices that are known not to have edges between them, but, as we discuss below, this will be implicit from the graph.) To determine a^t given q^t, the process $\mathcal{P}_b$ simply considers all graphs in the support of $\mathcal{D}_b$ that are consistent with K^{t-1}, and selects an answer with probability proportional to the number of such graphs that are also consistent with this answer.

We assume that before asking a query concerning a vertex v that is not in the knowledge graph, the algorithm performs a degree query q^t on v (which in particular determines whether v belongs to X^t, Y^t or Z^t). If the answer to this query is 2, then the algorithm performs two neighbor queries to get vs two neighbors. Also, if the algorithm gets a new vertex v as an answer to a neighbor query from a vertex $u \in Y^{t-1}$, then it performs a neighbor query on v so as to gets its second neighbor. A lower bound of T on the query complexity of the algorithm under theses restrictions translates into a lower bound of $\Omega(T)$ without these restrictions. Note that the algorithm need not ask any further degree queries, as their answers are determined by the knowledge graph (which is known to be isomorphic to a subgraph of G_0 or G_1). Also note that because for each degree-2 vertex that is introduced into the graph, the algorithm obtains both its neighbors, we may assume (without loss of generality) that the algorithm does not perform any vertex-pair queries (as their answers would always be negative).

We observe that for both processes, if the algorithm performs $o(n^{1/2})$ queries, then the probability that any answer to a degree query (on a new vertex) is not 2, is $o(1)$ (since $|Y_b| + |Z_b| = O(n^{1/2})$ for both $b = 0$ and $b = 1$). Let's refer to this "bad" event as E_1. In addition, by the second restriction on the algorithm (i.e., that each degree-2 vertex essentially "arrives with both its neighbors"), we have that for any neighbor query

q^t on a vertex in Y^{t-1}, the process $\mathcal{P}_b$ answers with a vertex (in X^t) whose label is uniformly selected among all $n - |V^{t-1}|$ "free" labels. That is, the distribution on these answers is identical under both processes.

It remains to analyze the answers to neighbor queries for vertices v in X^{t-1}. If v is the answer to a neighbor query from a vertex $u \in Y^{t-1}$, then each of the processes needs to select a single (additional) neighbor. Conditioned on this neighbor not belonging to V^{t-1}, for both processes, its label is uniformly distributed among all $n - |V^{t-1}|$ "free" labels. The same is true if v is introduced following a degree query and both its neighbors are determined not to belong to V^{t-1}. We denote the event that we get at least one neighbor that already belongs to the knowledge graph, by E_2. Observe that conditioned on E_1 and E_2 not occurring, the distributions on query-answer histories induced by the two processes, are identical (since, conditioned on these events, every degree query is answered by 2, and every neighbor query is answered by a uniformly selected new label).

It hence remains to upper bound the probability of the event E_2, under both processes, when the algorithm performs at most $n^{1/4}/c$ queries, for some sufficiently large constant c. Consider any prefix $(q^1, a^1), \ldots, (q^{t-1}, a^{t-1})$ of the query-answer history (where neither event E_1 or E_2 has yet occurred), and the corresponding knowledge graph K^{t-1}. Suppose that q^t is a degree query for a vertex v (that does not belong to V^{t-1}, and such that the answer to this query is 2). Consider first the process $\mathcal{P}_0$. Since for each pair of vertices in Y_0 there is a single vertex in X_0 that neighbors them, and there are at most $2(t-1)$ vertices in Y^{t-1} and at most $(t-1)$ in X^{t-1}, the probability (over the distribution induced by $\mathcal{P}_0$) that at least one of the neighbors of v belongs to Y^{t-1} is upper bounded by

$$\frac{2(t-1)\cdot(y_0-1)}{x_0-(t-1)} = \frac{2(t-1)\cdot(y_0-1)}{(y_0\cdot(y_0-1)/2)-(t-1)} \leq \frac{c't}{n^{1/2}}$$

for a constant c' (since $y_0 = \Theta(n^{1/2})$ and $t = O(n^{1/4})$). A similar (slightly lower) upper bound is obtained for $\mathcal{P}_0$ when v is the answer to a neighbor query from a vertex $u \in Y^{t-1}$ (since one of v's neighbors is already determined to be u).

Turning to $\mathcal{P}_1$, we get a similar upper bound. Namely, for a new vertex v (with degree 2), the probability that at least one of its neighbors belongs to Y^{t-1} is upper bounded by

$$\frac{2(t-1)\cdot(y_1/2)}{x_1-(t-1)} = \frac{2(t-1)\cdot(y_1/2)}{(y_1/2)^2-(t-1)} \leq \frac{c''t}{n^{1/2}}\ ,$$

for a constant c'', and a similar upper bound holds when v is an answer to a neighbor query from $u \in Y^{t-1}$. Summing over $t = 1, \ldots, T = n^{1/4}/c$, we get that the probability that E_2 occurs (for either processes) is a small constant, as required. □

B.2 A one-sided error lower bound for graphs with arboricity α

We now turn to a lower bound on non-constant arboricity graphs for one-sided error algorithms as stated in Theorem 7 in the introduction. We assume that $\alpha = O(n^{1/2})$ (recall that every graph with arboricity greater than $n^{1/2}$ must contain C_4s). The lower bound holds for $\alpha = \Omega(\log n)$. Note that the lower bound of Theorem 8 also holds for $\alpha = O(\log n)$ that is non-constant, since, as noted in the introduction we can add to the graphs in the lower-bound construction a subgraph with arboricity α that is C_4-free.

In order to prove Theorem 7, we define a distribution over n-vertex graphs for which the following holds. First, with high probability, a graph selected according to this distribution is $\Omega(1)$-far from being C_4-free. Second, the number of queries necessary to observe a C_4 with sufficiently high constant probability when the queries are answered by graph selected according to the distribution is $\Omega(n^{1/4}\alpha^{1/2})$. Furthermore, if a one-sided error algorithm performs less than $n^{1/4}\alpha^{1/2}/c$ queries for a sufficiently large c and does not observe a C_4, then it must accept, since the subgraph observed (including the degrees of the vertices) can be extended to a C_4-free graph.

The vertex set of each graph in the support of $\mathcal{D}$ is partitioned into five parts: X_1, X_2, Y_1, Y_2, and Z, where $|X_1| = |X_2| = n/4$, and for $d = n^{1/2}/c_2$, $|Y_1| = |Y_2| = (n\alpha)/(4d) = \Theta(n^{1/2}\alpha)$ and $|Z| =$

$n - (|X_1| + |X_2| + |Y_1| + |Y_2|)$. (We assume for simplicity that n is divisible by $2d$ and that α and d are even integer – the more general case can be handled similarly).

The distribution $\mathcal{D}$ is uniform over all graphs for which the following holds:

- There is a bipartite graph, denoted $G_{b,b'}$ between X_b and $Y_{b'}$ for $b, b' \in \{1, 2\}$, where vertices in X_b have $\alpha/2$ neighbors in $Y_{b'}$, vertices in $Y_{b'}$ have $d/2$ neighbors in X_b and vertices in Z have degree 0.
- There are no multiple edges.

Observe that there exists at least one such graph: Let each X_b consist of $\frac{n/4}{d/2}$ subsets, each of size $d/2$, and let each Y_b consist of $\frac{n\alpha/(4d)}{\alpha/2}$ subsets, each of size $\alpha/2$. For the edges, let there be a complete bipartite graph between the ith subset in each X_b to the ith subset of both Y_1 and Y_2. One can verify that indeed both conditions above hold. We stress that this graph is not a good candidate for a lower bound, as it contains many C_4s. However, as we prove below, when choosing uniformly over all graphs in the (non-empty) support of $\mathcal{D}$, detecting a C_4 requires $\Omega(n^{1/4}\alpha^{1/2})$ queries.

Lemma 4 With probability at least 9/10, a graph selected according to $\mathcal{D}$ is $\Omega(1)$-far from being C_4-free.

In order to establish Lemma 4, we define the following "bad events" (taken over the random choice of a graph according to $\mathcal{D}$). Let $\alpha' = \Theta(\alpha)$ (where the constant in the Θ notation will be determined subsequently). First, E^0 is the event that the fraction of edges between X_1 and Y_1 that participate in less than $\alpha'/2$ C_4s is larger than 1/2. Second, for each $b, b' \in \{1, 2\}$ and $\ell \geq 1$, let $D^\ell_{b,b'}$ be the set (bucket) of edges between X_b and $Y_{b'}$ that participate in a number of C_4s that is in the range $[(2^\ell + 1) \cdot \alpha', (2^{\ell+1} + 1) \cdot \alpha')$. The event $E^\ell_{b,b'}$ is the event that $|D^\ell_{b,b'}| > (2^{-2\ell}/c_3) \cdot |X_b| \cdot (\alpha/2)$ for some constant c_3.

Claim 5 For a sufficiently large $\alpha = \Omega(\log n)$, the probability that either E^0 occurs or some $E^\ell_{b,b'}$ occurs (for $b, b' \in \{1, 2\}$ and $\ell \geq 1$) is at most 1/10.

We prove Claim 5 momentarily, and first show how it implies Lemma 4.

Proof of Lemma 4: Assume that neither E^0 nor any $E^\ell_{b,b'}$ occur, which by Claim 5 holds with probability at least 9/10. Conditioned on E^0 not occurring, there are at least $|X_1| \cdot (\alpha/2) \cdot (\alpha'/2)$ $(= \Omega(n\alpha^2))$ C_4s in a graph selected according to $\mathcal{D}$. Suppose we mark the edges that participate in any one of the buckets $D^\ell_{b,b'}$ for $b, b' \in \{1, 2\}$ and $\ell \geq 1$ by "red" and all other edges by "green". By our assumption on the sizes of the buckets (the events $E^\ell_{b,b'}$ not occurring), the total number of C_4s that contain at least one red edge is upper bounded by

$$\sum_{b,b'} \sum_{\ell} |D^\ell_{b,b'}| \cdot (2^{\ell+1} + 1) \cdot \alpha' \leq 4 \cdot (2^{-2\ell}/c_3) \cdot |X_b| \cdot (\alpha/2) \cdot (2^{\ell+1} + 1) \cdot \alpha' \leq |X_1| \cdot (\alpha/2) \cdot (\alpha'/c'_3) \; . \tag{9}$$

By ensuring the $c'_3 \geq 4$, the number of C_4s residing on green edges (only) is at least

$$\frac{|X_1| \cdot \alpha \cdot \alpha'}{4} - \frac{|X_1| \cdot \alpha \cdot \alpha'}{8} = \Omega(n\alpha^2) \; . \tag{10}$$

Hence, with probability at least 9/10 over the choice of a graph according to $\mathcal{D}$, the following holds. There exists a set of green edges, such that the number of C_4s that reside on green edges is $\Omega(n\alpha^2)$, and each green edge participates in $O(\alpha)$ C_4s. This implies that there are $\Omega(n\alpha)$ edge-disjoint C_4s. To verify this consider running an iterative procedure, starting from the graph restricted to green edges. In each iteration, until there are no remaining C_4s, a C_4 is selected from the current graph and put in the set of edge-disjoint C_4s. Then all $O(\alpha)$ C_4s that include one of the edges in the selected C_4 are removed from the graph (i.e., their other edges are removed from the graph). Since we start with $\Omega(n\alpha^2)$ C_4s and in each iteration remove $O(\alpha)$ C_4s, the final set of edge-disjoint C_4s we get is of size $\Omega(n\alpha)$, implying that the graph is $\Omega(1)$-far from C_4-free. □

We next prove Claim 5.

Proof of Claim 5: We start by noting that in the selection of a random graph according to $\mathcal{D}$, the four graphs $G_{b,b'}$, $b, b' \in \{1,2\}$ are generated independently from each other (recall that $G_{b,b'}$ is a bipartite graph between X_b and $Y_{b'}$, where each vertex in X_b has $\alpha/2$ neighbors in $Y_{b'}$, and each vertex in $Y_{b'}$ has $d/2$ neighbors in X_b).

For simplicity of notations, let $b = 1$, $b' = 1$. The analysis below can be applied to any pair $b, b' \in \{1,2\}$. Fix any choice of bipartite graph $G_{2,2}$. We shall show that for any fixed edge (x,y) between $x \in X_1$ and $y \in Y_1$, the following holds over the random choice of $G_{1,2}$ and $G_{2,1}$. The expected number of C_4s that reside on (x,y) is α', and furthermore, with sufficiently high probability, the actual number does not deviate by too much from its expectation.

For a fixed edge $(x,y) \in X_1 \times Y_1$, Let $\Gamma_2(x)$ denote the set of $\alpha/2$ neighbors of x in Y_2 (as determined by the choice of $G_{1,2}$), and let $\Gamma_2(y)$ denote the set of $d/2$ neighbors of y in X_2 (as determined by the choice of $G_{2,1}$). Since we are considering a single vertex $x \in X_1$ and a single vertex $y \in Y_1$, these sets are uniformly distributed in Y_2 and X_1, respectively.

For $i \in [d/2]$ and $j \in [\alpha/2]$, let $\chi_{i,j}$ be a Bernoulli random variable that is 1 if there is an edge between the ith neighbor of y and the jth neighbor of x (and is 0 otherwise). Let $\mu = \Pr[\chi_{i,j}] = 1]$, so that

$$\mu = \frac{\alpha/2}{|Y_2|} = \Theta\left(\frac{n^{1/2}}{n}\right) = \Theta\left(\frac{1}{n^{1/2}}\right) . \tag{11}$$

Therefore, if we let $\alpha' = (d/2) \cdot (\alpha/2) \cdot \mu$ (so that $\alpha' = \Theta(\alpha)$), then

$$\mathbb{E}\left[\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]}} \chi_{i,j}\right] = \frac{d}{2} \cdot \frac{\alpha}{2} \cdot \mu = \alpha' . \tag{12}$$

We turn to analyze the variance of the sum.

$$\mathbb{V}\left[\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]}} \chi_{i,j}\right] = \mathbb{E}\left[\left(\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]}} \chi_{i,j}\right)^2\right] - \left(\mathbb{E}\left[\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]]}} \chi_{i,j}\right]\right)^2 \tag{13}$$

$$= \mathbb{E}\left[\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]}} (\chi_{i,j})^2\right] + \mathbb{E}\left[\sum_{\substack{i\in[d/2]\\ j_1\neq j_2\in[\alpha/2]}} \chi_{i,j_1}\chi_{i,j_2}\right] + \mathbb{E}\left[\sum_{\substack{i_1\neq i_2\in[d/2]\\ j\in[\alpha/2]}} \chi_{i_1,j}\chi_{i_2,j}\right]$$

$$+ \mathbb{E}\left[\sum_{\substack{i_1\neq i_2\in[d/2]\\ j_1\neq j_2\in[\alpha/2]}} \chi_{i_1,j_1}\chi_{i_2,j_2}\right] - \left(\frac{d}{2} \cdot \frac{\alpha}{2} \cdot \mu\right)^2 . \tag{14}$$

We analyze each of the terms separately. Since the $\chi_{i,j}$s are Bernoulli random variables, $(\chi_{i,j})^2 = \chi_{i,j}$ so that

$$\Pr[(\chi_{i,j})^2 = 1] = \Pr[\chi_{i,j} = 1] = \mu . \tag{15}$$

For every $i \in [d/2]$ and $j_1, j_2 \in [\alpha/2]$ such that $j_1 \neq j_2$,

$$\Pr[\chi_{i,j_1} = \chi_{i,j_2} = 1] \leq \left(\frac{\alpha/2}{|Y_2|}\right)^2 = \mu^2 , \tag{16}$$

and for every $i_1, i_2 \in [d/2]$ and $j \in [\alpha/2]$ such that $i_1 \neq i_2$,

$$\Pr[\chi_{i_1,j} = \chi_{i_2,j} = 1] \leq \left(\frac{d/2}{|X_2|}\right)^2 = \left(\frac{\alpha/2}{|Y_2|}\right)^2 = \mu^2 \,. \tag{17}$$

Finally, for $i_1 \neq i_2 \in [d/2]$ and $j_1 \neq j_2 \in [\alpha/2]$, we have that

$$\Pr[\chi_{i_1,j_1} = \chi_{i_2,j_2} = 1] = \Pr[\chi_{i_1,j_1} = 1] \cdot \Pr[\chi_{i_1,j_1} = 1 \,|\, \chi_{i_2,j_1} = 1] = \frac{\alpha/2}{|Y_2|} \cdot \frac{|X_2|(\alpha/2) - 1}{(|X_2| - 1)(|Y_2| - 1)} \,, \tag{18}$$

where the second term on the right-hand-side of Equation (18) is the fraction of pairs in $X_2 \times Y_2$ that have an edge between them, among all pairs with the exception of one pair that is an edge in $G_{2,2}$. It is not hard to verify that

$$\frac{|X_2|(\alpha/2) - 1}{(|X_2| - 1)(|Y_2| - 1)} \leq \frac{\alpha/2}{|Y_2|} \cdot \left(1 + \frac{2}{|Y_2|}\right) = \mu \cdot \left(1 + \frac{2}{|Y_2|}\right) \tag{19}$$

By combining Equations (14)–(19) (and recalling that $\alpha' = (d/2)(\alpha/2)\mu$), we get that

$$\begin{aligned}
\mathbb{V}\left[\sum_{\substack{i\in[d/2]\\ j\in[\alpha/2]}} \chi_{i,j}\right] &\leq \frac{d}{2}\cdot\frac{\alpha}{2}\cdot\mu + \frac{d}{2}\cdot\left(\frac{\alpha}{2}\right)^2\cdot\mu^2 + \left(\frac{d}{2}\right)^2\cdot\frac{\alpha}{2}\cdot\mu^2 \\
&\quad + \left(\frac{d}{2}\right)^2\cdot\left(\frac{\alpha}{2}\right)^2\cdot\mu^2\cdot\left(1+\frac{1}{|Y_2}\right) - \left(\frac{d}{2}\cdot\frac{\alpha}{2}\cdot\mu\right)^2 \\
&= \alpha'\cdot\left(1 + \frac{\alpha}{2}\cdot\mu + \frac{d}{2}\cdot\mu + \frac{d}{2}\cdot\frac{\alpha}{2}\cdot\frac{2}{|Y_2|}\cdot\mu\right) \\
&\leq 4\alpha' \,,
\end{aligned} \tag{20}$$

where in the last inequality we used the following facts: $\alpha \leq d$, $|Y_2| > \alpha$, and $(d/2)\mu \leq 1$.

By Chebishev's inequality, for any choice of $t > 0$,

$$\Pr\left[\left|\sum_{i\in[d/2],j\in[\alpha/2]} \chi_{i,j} - \alpha'\right| < t\cdot\alpha'\right] \leq \frac{\mathbb{V}\left[\sum_{\imath\in[d/2],j\in[\alpha/2]}\chi_{i,j}\right]}{t^2\cdot\left(\mathbb{E}\left[\sum_{i\in[d],j\in[\alpha]}\chi_{i,j}\right]\right)^2} \tag{21}$$

$$\leq \frac{4\alpha'}{t^2\cdot(\alpha')^2} = \frac{4}{t^2\cdot\alpha'} \tag{22}$$

By setting $t = 1/2$ we get that for any fixed edge $(x, y) \in X_1 \times Y_1$, the probability that (x, y) participates in less than $\alpha'/2$ C_4s is at most $16/\alpha'$. By Markov's inequality and the definition of the event E^0, we have that $\Pr[E^0] \leq 32/\alpha'$.

By applying the same argument to any other $b, b' \in \{1, 2\}$ (i.e., not necessary $b = b' = 1$), we get that for any fixed edge $(x, y) \in X_b \times Y_{b'}$ and for any choice of $t = 2^\ell$, $\ell \geq 0$, the probability that (x, y) participates in more than $(2^\ell + 1)\cdot\alpha'$ C_4s is at most $\frac{4}{2^{2\ell}\cdot\alpha'}$. Therefore, the expected number of such edges is at most $|X_b|\cdot(\alpha/2)\cdot\frac{4}{2^{2\ell}\cdot\alpha'}$. By Markov's inequality, the probability that there are more than $|X_b|\cdot(\alpha/2)\cdot\frac{1}{c_3\cdot 2^{2\ell}}$ such edges is at most $\frac{4c_3}{\alpha'}$.

By the definition of the bucket $D^\ell_{b,b'}$ (which contains a subset of the aforementioned edges) and the event $E^\ell_{b,b'}$, we have that $\Pr[E^\ell_{b,b'}] \leq \frac{4c_3}{\alpha'}$. Since $\alpha' = \Theta(\alpha)$ and $\alpha \geq c_1 \log n$, for a sufficiently large constant c_1, if we take a union bound over all bad events, we get a total probability of at most $1/10$, and the claim follows. □

Lemma 6 Let $\mathcal{A}$ be an algorithm that performs less than $n^{1/4}\alpha^{1/2}/c$ queries for a sufficiently large constant c. The probability that $\mathcal{A}$ detects a C_4 in a graph selected according to $\mathcal{D}$ is at most $1/10$.

Proof: Similarly to the proof of Theorem 8, consider a process $\mathcal{P}$ that answers the query of a testing algorithm while constructing a graph from $\mathrm{Sup}(\mathcal{D})$ (the support of $\mathcal{D}$) on the fly. Here too we define a knowledge graph K^t following the query-answer history $(q^1, a^1), \ldots, (q^t, a^t)$ and we require the algorithm to perform a degree query on each vertex v that is not yet in the knowledge graph before performing any other query involving it. The algorithm also receives the subset to which the new vertex belongs (as this is not be implied by the degree of the vertex). Let X_1^t, X_2^t, Y_1^t, Y_2^t and Z^t denote the corresponding subsets in the knowledge graph K^t.

The set of edges in K^t is denoted by E^t, where for each $e \in E^t$ there are two associated labels, $\phi_X^t(e)$ for its endpoint in $X^t = X_1^t \cup X_2^t$, and $\phi_Y^t(e)$ for its endpoint in $Y^t = Y_1^t \cup Y_2^t$. We assume that a neighbor query $(x_{b,i}, j)$ is answered by a pair $(y_{b',i'}, j')$ where $y_{b',i'}$ is the jth neighbor of $x_{b,i}$ and $x_{b,i}$ is the j'th neighbor of $y_{b',i'}$. Similarly, if a vertex-pair query $(x_{b,i}, y_{b',i'})$ is answered positively, then the algorithm also gets the corresponding edge labels ϕ_X and ϕ_Y for this edge. The knowledge graph also includes a set $\overline{E}^t$ of non-edges (corresponding to negative answers to vertex-pair queries).

Similarly to the proof of Theorem 8, in order to determine a^t given q^t, the process $\mathcal{P}$ considers all graphs $G \in \mathrm{Sup}(\mathcal{D})$ that are consistent with K^{t-1}, It then selects an answer with probability proportional to the number of such graphs G for which G is also consistent with this answer.

Consider any knowledge graph K^{t-1} for $t < n^{1/4}\alpha^{1/2}/c$. We would like to bound the probability that either a neighbor query or a vertex-pair query adds to the knowledge graph an edge (x, y) that creates a C_4 in the knowledge graph. We shall actually bound the probability that a neighbor query is answered with some vertex that already belongs to the knowledge graph or a vertex-pair query is answered positively (even if this does not reveal a C_4). We shall refer to such an answer as a collision.

Let $\mathcal{G}(K^{t-1})$ denote the subset of graphs $G \in \mathrm{Sup}(\mathcal{D})$ such that G is consistent with K^{t-1}. For a query q^t (either a neighbor query or a pair query), let $\mathcal{G}_c(K^{t-1}, q^t)$ denote the subset of graphs $G \in \mathcal{G}(K^{t-1})$ for which the answer to q^t according to G results in a collision, and similarly define $\mathcal{G}_{nc}(K^{t-1}, q^t)$ for the case in which the answer is not a collision.

Subclaim Let K^{t-1} be any knowledge graph where $t < n^{1/4}\alpha^{1/2}/c$, and let q^t be a neighbor or pair query. Then

$$\frac{|\mathcal{G}_{nc}(K^{t-1}, q^t)|}{|\mathcal{G}_c(K^{t-1}, q^t)|} = \Omega(n^{1/4}\alpha^{1/2}) \ .$$

Proof: In order to prove the subclaim we define an auxiliary bipartite graph $\mathcal{B}(K^{t-1}, q^t)$. Each node in $\mathcal{B}(K^{t-1}, q^t)$ corresponds to a graph $G \in \mathcal{G}(K^{t-1})$. Nodes on the left side correspond to graphs in $\mathcal{G}_c(K^{t-1}, q^t)$, and nodes on the right side to graphs in $\mathcal{G}_{nc}(K^{t-1}, q^t)$. There is an edge between two nodes corresponding to graphs G_1 and G_2, if G_2 can be obtained from G_1 by a certain "swap" modification (which will be explained precisely below). We shall show that the number of nodes on the left is smaller by a factor of $\Omega(n^{1/4}\alpha^{1/2})$ than the number of nodes on the left by appropriately lower bounding the degree of the former and upper bounding the degree of the latter. From this point on we use the shorthand notations $\mathcal{B}$, $\mathcal{G}$, $\mathcal{G}_c$ and $\mathcal{G}_{nc}$ for $\mathcal{B}(K^{t-1}, q^t)$, $\mathcal{G}(K^{t-1})$, $\mathcal{G}_c(K^{t-1}, q^t)$ and $\mathcal{G}_{nc}(K^{t-1}, q^t)$, respectively.

For a graph $G \in \mathcal{G}$ and an edge e in G we shall use the notation $\phi_X^G(e)$ and $\phi_Y^G(e)$ for its label with respect to its endpoint in X and in Y, respectively. Let q^t be a neighbor query $nbr(v, i)$ where $v \in X^{t-1}$, and let G_c be a graph in $\mathcal{G}$ for which the answer to q^t results in a collision. That is, the answer a^t is (u, j) for some $u \in Y^{t-1}$. The neighbors of node G_c in $\mathcal{G}_{nc}$ are nodes representing graphs that can be obtained from G_c by swapping the edge (v, u) with a different edge (v, u') where $u' \notin Y^{t-1}$, and such that the resulting graph belongs to $\mathcal{G}$. We prove that there are $\Omega(n^{1/2}\alpha)$ such graphs.

In order to modify the graph $G_c \in \mathcal{G}_c$ to obtain a graph $G_{nc} \in \mathcal{G}_{nc}$, we proceed as follows. We choose a vertex $u' \in Y \setminus Y^{t-1}$ (where the constraints on this choice are explained momentarily), and let v' denote the jth neighbor of u' in the graph G_c. Note that v' is in X. Let $i' = \phi_X^{G_c}((v', u'))$. Now we swap the pair of edges (v, u) and (v', u') with the pair $(v, u'), (v', u)$ while maintaining the original edge-labels. That is, $\phi_X^{G_{nc}}((v, u')) = i, \phi_Y^{G_{nc}}((v, u')) = j$ and $\phi_X^{G_{nc}}((v', u)) = i', \phi_Y^{G_{nc}}((v, u')) = j'$. Since $u' \notin Y^{t-1}$, if G_{nc} is in $\mathcal{G}$, then it is in $\mathcal{G}_{nc}$. In order to ensure that $G_{nc} \in \mathcal{G}$, it is sufficient (and necessary), that there is no edge in G_c between v and u' and no edge between v' and u and that $(v', u) \notin \overline{E}^{t-1}$.

Observe that the selection of u' uniquely determines v'. There are $|Y^{t-1}| \leq t = O(n^{1/4}\alpha^{1/2})$ "illegal" choices of $u' \in Y^{t-1}$ and $2\alpha = O(n^{1/4}\alpha^{1/2})$ illegal choices of u' that are neighbors of v. There are at most t choices of v' such that $(v', u) \in \overline{E}^{t-1}$, and d choices of neighbors of u. For each of these $d + t$ choices of v' there are at most 2α neighbors u' such that v' may be the jth neighbor of u'. This rules out at most $2(d + t) \cdot \alpha$ choices of u'. Since $|Y| = |X|\alpha/d = n\alpha/(2d)$ and $d = n^{1/2}/c_2$, for a sufficiently large constant c_2, the number of choices of u' for which $G_{nc} \in \mathcal{G}$ is $\Omega(n^{1/2}\alpha)$.

We now upper bound the number of neighbors of each node in $\mathcal{B}$ that corresponds to a graph $G_{nc} \in \mathcal{G}_{nc}$. Assume that the answer to $q^t = nbr(v, i)$ in G_{nc} is $a^t = (u', j')$. That is, u' is the ith neighbor of v in G_{nc}, and v is the j'th neighbor of u'. By the definition of $\mathcal{B}$, every neighbor of G_{nc} in $\mathcal{B}$ is a graph G_c where the ith neighbor of v is a vertex $u'' \in Y^{t-1}$. Furthermore, by the definition of the swap process, if the edge $(v, u'') \in G_c$ was replaced by a swap process and resulted in an edge (v, u') where $\phi_Y^{G_{nc}}((v, u')) = j'$, it must be the case that $\phi_Y^{G_c}((v, u'')) = j'$ (since the swapping process involves only two edges such that both have the same ϕ_Y value). Hence, for each choice of $u'' \in Y^{t-1}$, the graph G_c from which G_{nc} was obtained by a swap process is uniquely defined: Let v'' denote the j'th neighbor of u'' in G_c. The graph G_c is a graph where (v, u'') is an edge with $\phi_X^{G_c}((v, u'')) = i, \phi_Y^{G_c}((v, u'')) = j'$ and (v'', u') is an edge with $\phi_Y^{G_c}((v'', u')) = j'$ (and $\phi_X^{G_c}((v'', u')) = i''$ for some i''). Since there are at most $|Y^{t-1}| = O(n^{1/4}\alpha^{1/2})$ options to choose u'', it holds that G_{nc} has $O(n^{1/4}\alpha^{1/2})$ neighbors in $\mathcal{B}$ (it might be the case that some of the resulting graphs G_c are not in $\mathcal{G}$ but the upper bound still holds).

Since each node corresponding to a graph in $\mathcal{G}_c$ has $\Omega(n^{1/2}\alpha)$ neighbors and each node corresponding to a graph in $\mathcal{G}_{nc}$ has $O(n^{1/4}\alpha^{1/2})$ neighbors, the subgraph follows for any neighbor query $nbr(v, i)$ for $v \in X^{t-1}$.

We next consider the case in which $q^t = nbr(u, j)$ for $u \in Y^{t-1}$. The swap operation and analysis are very similar to the case in which $q^t = nbr(v, i)$ for $v \in X^{t-1}$, and so we emphasize only the differences. Let $G_c \in \mathcal{G}_c$ be a graph for which q^t is answered by (v, i) where $v \in X^{t-1}$. To perform a swap operation and obtain a graph in $\mathcal{G}_{nc}$ we select a vertex $v' \notin X^{t-1}$ such that v' is not a neighbor of u in G_c, and for the ith neighbor of v', denoted u' we have that u' is not a neighbor of v in G_c, but $(u', v) \notin \overline{E}^{t-1}$. For each such legal choice of v' we can perform a swap operation between the edges (u, v) and (u', v') while maintaining the edge labels as in the case that $q^t = nbr(v, i)$ for $v \in X^{t-1}$. The constraints on v' rule out at most $t + 2d + (t + \alpha) \cdot 2d$ vertices, and so the number of legal choices of v' is $\Omega(|X|) = \Omega(n)$. On the other hand, for any graph $G_{nc} \in \mathcal{G}_{nc}$, for which the answer to $q^t = nbr(v, j)$ is (v', i') for $v' \notin X^{t-1}$ there are at most t choices of $v'' \in X^{t-1}$ such that a swap operation between the edges (v'', u) and (v', u') resulted in the graph G_{nc}. Hence, here the ratio between $|\mathcal{G}_{nc}|$ and $|\mathcal{G}_c|$ is $\Omega(n/t) = \Omega(n^{3/4}/\alpha^{1/2}) = \Omega(n^{1/4}\alpha^{1/2})$, as required.

Finally we turn to the case in which q^t is a pair query between two vertices $v \in X^{t-1}$ and $u \in Y^{t-1}$. Consider a graph $G_c \in \mathcal{G}_c$ for which the answer is positive. We can perform a swap with any edge (v', u') in G_c such that $(v', u') \notin E(K^{t-1})$ and in addition, $(v', u) \notin E(G_c)$, $(v, u') \notin E(G_c)$ and $(v', u), (v, u') \notin \overline{E}^{t-1}$. The number of edges in G_c on which a swap cannot performed with (u, v) is hence upper bounded by $t + (d + t) \cdot \alpha + d \cdot (t + \alpha) = O(d\alpha)$. This leaves $\Omega(n\alpha)$ edges with which a swap can be performed (each resulting in a different graph in $\mathcal{G}_{nc}$). On the other hand, for a graph $G_{nc} \in \mathcal{G}_{nc}$, for which there is no edge between u and v, the number of graphs in $\mathcal{G}_c$ for which a swap operation resulted in G_{nc} is at most $d\alpha$ (the number of pairs of vertices (u', v') such that $(u, v'), (u', v) \in E(G_{nc})$). Hence the ratio between $|\mathcal{G}_{nc}|$ and $|\mathcal{G}_c|$ is $\Omega((n\alpha)/(d\alpha)) = \Omega(n/d) = \Omega(n^{1/2}) = \Omega(n^{1/4}\alpha^{1/2})$. ◊

By the above, the probability that the algorithm detects a C_4 following any one of its at most $n^{1/4}\alpha^{1/2}/c$ queries is upper bounded by a small constant (e.g., 1/10). □

Theorem 7 follows by combining Lemmas 4 and 6.

C Results for C_5

The results for C_5 when the graph has constant arboricity are very similar to those for C_4 (when the arboricity α is a constant), and hence we only provide a sketch in this section.

We start with the upper bound (as stated in Theorem 1 in the introduction).

Theorem 9 There exists a one-sided error algorithm for testing C_5-freeness in graphs with arboricity at most $\alpha = O(1)$ over n vertices whose query complexity is $\widetilde{O}(n^{1/4}/\epsilon^4)$.

The algorithm is an adaptation of the algorithm for testing C_4-freeness. While the algorithm for testing C_4-freeness attempted to find a C_4 by obtaining two length-2 edge-disjoint paths from a vertex v to a vertex v', the algorithm described next attempts to find a C_5 by obtaining two edge-disjoint paths from a vertex v to a vertex v' such that one of these paths is of length 2 and the other of length 3.

Test-C_5-freeness(n, ϵ, α)

1. Let $\theta_0 = 4\alpha/\epsilon$ and $\theta_1 = c_1\sqrt{n}/\epsilon$ (for a sufficiently large constant c_1).
2. Repeat the following $t = \Theta(1/\epsilon)$ times:
   (a) Select an edge e by calling the procedure Select-an-edge(α, ϵ) (as provided in Section 3). If it does not return an edge, then continue to the next iteration.
   (b) Select an endpoint v of e by flipping a fair coin.
   (c) If $d(v) \leq \theta_1$, then select $s_1 = \Theta(\sqrt{d(v)/\epsilon})$ random neighbors of v, and for each neighbor u such that $d(u) \leq \theta_0$, query all the neighbors of u, and all the neighbors u' of u for which $d(u) \leq \theta_0$. That is, from each of the s_1 selected neighbors of v, perform a restricted BFS of depth 2, where only vertices with degree at most θ_0 are further explored.
   (d) Otherwise ($d(v) > \theta_1$), perform $s_2 = \Theta(\epsilon^{-3} \cdot \sqrt{n \log n/\theta_1})$ $(= \tilde{\Theta}(n^{1/4}/\epsilon^3))$ random walks of length 3 starting from v.
   (e) If a C_5 is detected, then return it, 'Reject' and terminate.
3. Return 'Accept'.

Since the algorithm rejects a graph only if it observes a C_5, it always accepts C_5-free graphs. Hence, consider a graph G that is ϵ-far from being C_5-free. Similarly to the analysis of the algorithm for testing C_4-freeness, we let $\mathcal{C}$ be a maximal set of edge-disjoint C_5s such that no C_5 in this set contains an edge between two vertices with degree greater than θ_0. We then define $\mathcal{C}_1$ and $\mathcal{C}_2$ to be the subsets of $\mathcal{C}$ containing C_5s with no edge between two vertices with degree greater than θ_0 and at most one (respectively, exactly two) vertices with degree greater then θ_1. Here too, either $|\mathcal{C}_1| \geq \epsilon m/16$ or $|\mathcal{C}_2| \geq \epsilon m/16$.

If $|\mathcal{C}_1| \geq \epsilon m/16$, then by applying a coloring process as in the proof of Theorem 6, we obtain a set $\mathcal{C}'_1 \subseteq \mathcal{C}_1$ in which every vertex v participating in some C_5 in $\mathcal{C}'_1$ actually participates in $\Omega(\epsilon d(v))$ such C_5s. Since every 5-cycle $\rho \in \mathcal{C}'_1$ contains at most one vertex of degree greater than θ_1, we can label the vertices in ρ by $v_i(\rho)$, $i = 0, 1, \ldots, 4$ where $v_2(\rho)$ is the highest degree vertex, such that the following holds. First, $v_0(\rho) \leq \theta_1$, and second, $v_1(\rho)$, $v_3(\rho)$ and $v_4(\rho)$ all have degree at most θ_0. This implies that if the algorithm selects a vertex $v \in V_0(\mathcal{C}'_0) = \bigcup_{\rho \in \mathcal{C}'_1}\{v_0(\rho)\}$ (which occurs with high constant probability), then with high constant probability the algorithm will detect a C_5 in Step 2c.

If $|\mathcal{C}_2| \geq \epsilon m/4$, then we apply a similar coloring process to obtain a set $\mathcal{C}'_1 \subseteq \mathcal{C}_2$ in which every vertex v participating in some C_5 in $\mathcal{C}'_2$ actually participates in $\Omega(\epsilon d(v))$ such C_5s. Since every 5-cycle $\rho \in \mathcal{C}'_2$ contains two vertices, v and v' of degree greater than θ_1, there is one length-2 path between v and v' that passes through a vertex with degree at most θ_0, and one length-3 path that passes through two vertices with degree at most θ_0. An analysis similar to the given in the proof of Claim 3 shows that in this case (that $|\mathcal{C}_2| \geq \epsilon m/4$), with high constant probability, a C_5 will be detected in Step 2d of the algorithm.

We now turn to the lower bound.

Theorem 10 Testing C_5-freeness in constant-arboricity graphs over n vertices requires $\Omega(n^{1/4})$ queries for constant ϵ.

The high-level idea of the proof of Theorem 10 is very similar to that of Theorem 8. The main difference is that in both G_0 and G_1 (based on which the distributions $\mathcal{D}_0$ and $\mathcal{D}_1$ are defined), some of the vertices in

X_b (for $b \in \{0,1\}$) are connected by an edge (so that the graphs are not bipartite), and for each such edge (v_1, v_2), there are two vertices u_1 and u_2 in Y_b, such that u_1 neighbors v_1 and u_2 neighbors v_2.

In G_0, every pair of vertices u_1, u_2 in Y_0 either neighbor a unique vertex v_0 in X_0 or there is a unique edge (v_1, v_2) between a pair of vertices in X_0 such that u_1 neighbors v_1 and u_2 neighbors v_2. On the other hand, in G_1, edge-disjoint C_5s are created by having the same pair u_1, u_2 in Y_1 neighbor both a single vertex v_0 and the endpoints of an edge (v_1, v_2) as described above, so that $(v_0, u_1, v_1, v_2, u_2)$ is a C_5.

D An upper bound for C_6

In this section we prove Theorem 11, which is restated next.

Theorem 11 There exists a one-sided error algorithm for testing C_6-freeness in graphs of constant arboricity whose query complexity and running time are $\tilde{O}(n^{1/2} \cdot \mathrm{poly}(1/\epsilon))$.

The algorithm referred to in Theorem 11 is presented next.

Test-C_6-freeness(n, ϵ, α)

1. Let $\theta_0 = 4\alpha/\epsilon$, and $\theta_1 = c_1 \cdot n^{1/2} \log^2 n/\epsilon^2$ (for a sufficiently large constant c_1).
2. Repeat the following $t = \mathrm{poly}(\log n/\epsilon)$ times:
   (a) Select a vertex v uniformly at random and query its degree. If $d(v) > \theta_0$, then continue to the next iteration.
   (b) Perform a restricted BFS starting from v to depth 4, where the BFS is restricted in the following sense.
      i. Whenever a vertex u is reached such that $d(u) \leq \theta_0$, all its neighbors are queried.
      ii. Whenever a vertex u is reached such that $d(u) > \theta_0$ and u is reached from a vertex u' such that $d(u') \leq \theta_0$, there are two sub-cases. If $d(u) \leq \theta_1$, then all of u's neighbors are queried. Otherwise, θ_1 neighbors of u are selected uniformly at random.
      iii. Whenever a vertex u is reached from a vertex u' such that both $d(u) > \theta_0$ and $d(u') > \theta_0$, the BFS does not continue from u.
   (c) If a C_6 is detected, then return 'Reject' (and terminate).
3. Return 'Accept'.

Since the algorithm rejects only when it detects a C_6, it always accepts when the graph G is C_6-free, and hence, from this point on, we focus on the case that G is ϵ-far from C_6-free. Let $\mathcal{C}$ be a maximal set of edge-disjoint C_6s in G where for every C_6 in $\mathcal{C}$, there are no edges in which both endpoints have degree greater than θ_0 (where $\theta_0 = 4\alpha/\epsilon$, as defined in the algorithm). Since the arboricity of G is at most α, we have that $|\mathcal{C}| \geq \epsilon m/12$.

Let L denote the set of vertices with degree at most θ_0, let M denote the vertices with degree greater than θ_0, and let H denote those with degree greater than θ_1 (where L stands for low, M for medium, and H for high). We partition the C_6s in $\mathcal{C}$ into three subsets: $\mathcal{C}_1$ contains those with at most one vertex in M, and for $b \in \{2,3\}$, $\mathcal{C}_b$ contains those with exactly b vertices in M. For each $b \in \{1,2,3\}$, we consider the case that $|\mathcal{C}_b| \geq \epsilon m/36$, where for at least one of the three this lower bound should hold.

Claim 7 If $|\mathcal{C}_1| \geq \epsilon m/36$, then the algorithm will detect a C_6 with probability at least 2/3.

Proof: Consider the following set of useful vertices U. For each $\rho \in \mathcal{C}_1$, if all vertices on ρ belong to L, then they all belong to U. If there is a vertex u on ρ that belongs to M, then the vertex $v \in L$ at distance 3 from u (on ρ), belongs to U (that is, the vertex v that is opposite to u on ρ). Since $|\mathcal{C}_1| \geq \epsilon m/36$, and each vertex in L belongs to at most $\theta_0/2$ edge-disjoint C_6s in $\mathcal{C}_1$, we have that $|U| = \Omega(\epsilon n)$. Therefore, with probability at least 2/3, some vertex $v \in U$ is selected in at least one of the iterations of the algorithm. Conditioned on this event, the restricted BFS starting from v will detect a C_6. □

Claim 8 If $|\mathcal{C}_2| \geq \epsilon m/36$, then the algorithm will detect a C_6 with probability at least 2/3.

Proof: Recall that $\mathcal{C}_2$ is the set of C_6s in $\mathcal{C}$ with exactly two vertices in M, i.e., that have degree greater than θ_0. We consider several sub-cases depending on the number of these vertices that are also in H. Let $\mathcal{C}_{2,1}$ consists of those C_6s in $\mathcal{C}_2$ that contain at most one vertex in H, and $\mathcal{C}_{2,2}$ those that contain two vertices in H, and note that $\mathcal{C}_2 = \mathcal{C}_{2,1} \uplus \mathcal{C}_{2,2}$ so that at least one of these subsets is of size at least $|\mathcal{C}_2|/2$.

If $|\mathcal{C}_{2,1}| \geq |\mathcal{C}_2|/2$, then let U denote those vertices in L that reside on C_6s in $\mathcal{C}_{2,1}$ that either do not have any vertex in H, or are at distance at least two from the single vertex in H on the C_6. Similarly to the proof of Claim 7, the algorithm selects a vertex from U with probability at least 2/3, and conditioned on this event, a C_6 is detected.

We now turn to the case that $|\mathcal{C}_{2,2}| \geq |\mathcal{C}_2|/2$. We further partition the C_6s in $\mathcal{C}_{2,2}$ according to the distance on the C_6 between the two vertices in H, which is either 2 or 3, and denote the two subsets by $\mathcal{C}_{2,2,2}$ and $\mathcal{C}_{2,2,3}$, respectively. In what follows we analyze the case that $|\mathcal{C}_{2,2,2}| \geq |\mathcal{C}_{2,2}|/2$. The case that $|\mathcal{C}_{2,2,3}| \geq |\mathcal{C}_{2,2}|/2$, is analyzed similarly.

For each pair of vertices $u, u' \in H$, let $\mathcal{C}(u,u')$ denote the subset of C_6s in $\mathcal{C}_{2,2,2}$ on which u and u' reside. We shall say that the set $\mathcal{C}(u,u')$ is useful if $|\mathcal{C}(u,u')| \geq \frac{(\epsilon/600)\cdot\max\{d(u),d(u')\}}{|H|}$, otherwise it is un-useful. By this definition, the number of C_6s in $\mathcal{C}_{2,2,2}$ that belong to un-useful sets is at most

$$\sum_{u\in H}\sum_{u'\in H: d(u')\leq d(u)} \frac{(\epsilon/600)\cdot d(u)}{|H|} \leq \epsilon m/300 \,. \tag{23}$$

Let $\mathcal{C}'_{2,2,2}$ denote the subset of C_6s in $\mathcal{C}_{2,2,2}$ that belong to useful sets, so that $|\mathcal{C}'_{2,2,2}| \geq \epsilon m/300$.

The algorithm "tries" to detect C_6s of this form by starting from a vertex v that is at distance 2 from both u and u' on some C_6 in $\mathcal{C}(u,u')$, and obtaining a path of length 2 and a path of length 4 that collide on either u or u'. Assume w.l.o.g. that $d(u') \leq d(u)$. Recall that except for u and u', all other vertices on the C_6s in $\mathcal{C}(u,u')$ are in L. Therefore, a restricted BFS starting from v will surely reach both u and u' at depth 2. Then, for the algorithm to reach u from u' in two more steps, it suffices that one of their common neighbors in L is selected in Step 2(b)ii among the θ_1 sampled random neighbors of u'. In such a case the restricted BFS will surely reach u via u', and thus obtain a C_6. Since the number of common neighbors of u and u' is at last $|\mathcal{C}(u,u')|$, the probability of this event is at least

$$1 - \left(1 - \frac{(\epsilon/600)\cdot\max\{d(u),d(u')\}}{|H|\cdot\min\{d(u),d(u')\}}\right)^{\theta_1} \,.$$

Since $|H| \leq 2m/\theta_1$, by the setting of θ_1 (for a sufficiently large constant c_1 and recalling that the graph has constant arboricity, so that $m = O(n)$), this probability is at least 9/10. Combining this with the probability of the selection of some vertex v in L as defined above (which occurs with probability $\Omega(\epsilon^2)$), the claim follows. □

Claim 9 If $|\mathcal{C}_3| \geq \epsilon m/36$, then the algorithm will detect a C_6 with probability at least 2/3.

Proof: Recall that by the definition of $\mathcal{C}_3$, for each C_6 in $\mathcal{C}_3$ there are three vertices that belong to M, and three that belong to L, where each vertex in M neighbors two vertices in L. We start by selecting a subset of $\mathcal{C}_3$, denoted $\mathcal{C}'_3$, such that each vertex in L belongs to at most a single C_6 in $\mathcal{C}'_3$. Since each vertex in L has degree at most $\theta_0 = O(1/\epsilon)$, if we select $\mathcal{C}'_3$ in a simple iterative greedy manner, then we get that $|\mathcal{C}'_3| \geq \epsilon_3 m$, where $\epsilon_3 = \Omega(\epsilon^2)$. Let $M' \subseteq M$ be the subset of vertices in M that reside on C_6s in $\mathcal{C}'_3$ and let $L' \subseteq L$ be the corresponding subset of vertices in L. Let the edges on the C_6s in $\mathcal{C}'_3$ be colored green in G.

We next define an auxiliary multi-graph G' where (some of the) triangles in G' correspond to C_6s in G. For each vertex in M' we have a vertex in G'. In addition, there is a special ground vertex, which we denote by v_g. For each pair of vertices, $u_1, u_2 \in M'$ and for each length-2 green path (u_1, v, u_2) (where $v \in L'$), there is an edge in G'. We think of this edge as being colored blue in G' and labeled by the vertex v in L'. In addition, for each $u \in M'$, and each non-green incident edge that it has in G, there is an ("uncolored") edge between u and v_g. By this construction, there are $|\mathcal{C}'_3|$ blue edge-disjoint triangles in G' (and the degree of every vertex $u \in M'$ in G' is the same as its degree in G).

Claim 10 Let G' be a multigraph over $m' \leq m$ edges, with m'_b blue edges, and at least $\epsilon_3 m$ edge-disjoint blue triangles. Let D be any distribution over edges in G' such that for each blue edge in G', the probability that it is selected according to D is $\Omega(1/m'_b)$. Consider the following process. First we select a blue edge (u, u') according to D. Next, for each of u and u', if its degree is at most θ_1, then we query all its neighbors in G', and otherwise we select θ_1 random neighbors in G'. The probability that the process obtains a triangle (u, u', u'') in G' where $u'' \in M'$ is $\Omega(\epsilon_3/\log^4 n)$.

Proof: Let $\mathcal{T}$ denote a maximal set of edge-disjoint triangles in G' whose edges are all colored blue, so that $|\mathcal{T}| \geq \epsilon_3 m$. Let $\mathcal{T}_1$ denote the subset of triangles in $\mathcal{T}$ that contain at most one vertex in H, and let $\mathcal{T}_2$ denote those that contain at least two vertices in H. If $|\mathcal{T}_1| \geq |\mathcal{T}|/2$, then we are done: conditioned on selecting an edge between two vertices in $M' \setminus H$ that belongs to a triangle in $\mathcal{T}_1$, the procedure will detect this triangle, as it queries all neighbors of the selected edge's endpoints, and the probability of selecting such an edge is $\Omega(\epsilon_3)$.

From this point on we address the case that $|\mathcal{T}_2| \geq |\mathcal{T}|/2$. For a vertex $u \in M'$ we use $d(u)$ to denote its degree in G'. For each pair of vertices $u_1, u_2 \in M'$, let $E(u_1, u_2)$ denote the set of edges between u_1 and u_2 that reside on triangles in $\mathcal{T}_2$ (so that in particular, they are all blue), and let $e(u_1, u_2) = |E(u_1, u_2)|$. Suppose the procedure described in the claim selects an edge $(a, b) \in E(u_1, u_2)$ where $a, b \in H$. Recall that in such a case, θ_1 random neighbors of a and θ_1 random neighbors of b are sampled. For $i \in [\theta_1]$ and $j \in [\theta_1]$, let $\chi_{i,j}(a, b)$ be a Bernoulli random variable that is 1 if and only if the ith randomly selected neighbor of a equals the jth randomly selected neighbor of b, where this neighbor is not v_g (thus detecting a blue triangle). Note that by this definition, the expected number of triangles detected (conditioned on selecting (a, b)) is $\sum_{i,j \in [\theta_1]} \chi_{i,j}(a, b)$, where

$$\Pr[\chi_{i,j}(a, b) = 1] = \sum_{c \in M} \frac{e(a, c)}{d(a)} \cdot \frac{e(b, c)}{d(b)} \ . \tag{24}$$

A simple but important observation is that, since each of the $e(a, b)$ edges in $E(a, b)$ participates in some triangle (a, b, c) in $\mathcal{T}_2$ where these triangles are edge-disjoint, $e(a, b)$ is upper bounded by $\sum_{c \in M} \min\{e(a, c), e(b, c)\}$, which in turn is upper bounded by $\sum_{c \in M} e(a, c) \cdot e(b, c)$. That is,

$$\sum_{c \in M} e(a, c) \cdot e(b, c) \geq \sum_{c \in M} \min\{e(a, c), e(b, c)\} \geq e(a, b) \ .$$

Therefore,

$$\Pr[\chi_{i,j}(a, b) = 1] \geq \sum_{c \in M} \frac{e(a, b)}{d(a) \cdot d(b)} \ . \tag{25}$$

We next show that there exists a relatively large set of edges between pairs (a, b) where $a, b \in H$, for which $\Pr[\chi_{i,j}(a, b) = 1] = \tilde{\Omega}(1/n)$. We shall refer to such pairs a, b as useful pairs. This will imply that the expected number of triangles detected when selecting an edge between one of these pairs is $\tilde{\Omega}(\theta_1^2/n)$. By the setting of θ_1, this is greater than 1. Our definition of this set of useful pairs will also ensure that the variance is bounded, allowing us to deduce that a triangle is detected with sufficiently high probability. Details follow.

We start by partitioning the set $\mathcal{T}_2$ into subsets as follows. First, for each triangle $\Delta \in \mathcal{T}_2$, we denote it by $(a_\Delta, b_\Delta, c_\Delta)$, where, without loss of generality, $a_\Delta, b_\Delta \in H$. For every four integers $x_A, x_B, y_{A,C}, y_{B,C}$, the subset $\mathcal{T}_2(x_A, x_B, y_{A,C}, y_{B,C})$ consists of all triangles $\Delta \in \mathcal{T}_2$ for which the following holds. $d(a_\Delta) \in [2^{x_A - 1} \cdot \theta_1, 2^{x_A} \cdot \theta_1)$, $d(b_\Delta) \in [2^{x_B - 1} \cdot \theta_1, 2^{x_B} \cdot \theta_1)$, $e(a_\Delta, c_\Delta) \in [2^{y_{A,C} - 1}, 2^{y_{A,C}})$, and $e(b_\Delta, c_\Delta) \in [2^{y_{B,C} - 1}, 2^{y_{B,C}})$. Let $\mathcal{T}_2^* = \mathcal{T}_2(x_A^*, x_B^*, y_{A,C}^*, y_{B,C}^*)$ be the largest subset, so that $|\mathcal{T}_2^*| \geq \epsilon_3 m/\log^4 n$. Let $A^* \stackrel{\text{def}}{=} \{a_\Delta : \Delta \in \mathcal{T}_2^*\}$ and $B^* \stackrel{\text{def}}{=} \{b_\Delta : \Delta \in \mathcal{T}_2^*\}$. For each pair of vertices $a \in A^*$ and $b \in B^*$, let $E^*(a, b)$ be the subset of edges between a and b such that there exists $\Delta \in \mathcal{T}_2^*$ for which $a = a_\Delta$ and $b = b_\Delta$, let $e^*(a, b) = |E^*(a, b)|$ and let $C^*_{a,b} \stackrel{\text{def}}{=} \{c_\Delta : a = a_\Delta, b = b_\Delta, \Delta \in \mathcal{T}_2^*\}$. We have:

$$s^* \stackrel{\text{def}}{=} \sum_{a \in A^*, b \in B^*} e^*(a, b) \geq \frac{\epsilon_3 m}{\log^4 n} \ , \tag{26}$$

and for every $a \in A^*$ and $b \in B^*$, similarly to what was argued before regarding $e(a,b)$,

$$\sum_{c \in C^*(a,b)} e(a,c) \cdot e(b,c) \geq e^*(a,b) \ . \tag{27}$$

Let

$$P^* \stackrel{\text{def}}{=} \left\{ (a,b) \ : \ a \in A^*, b \in B^*, e^*(a,b) \geq \frac{s^*}{2 \cdot |A^*| \cdot |B^*|} \right\} \ . \tag{28}$$

Since

$$\sum_{(a,b) \in A^* \times B^* \setminus P^*} e^*(a,b) \leq |A^*| \cdot |B^*| \cdot \frac{s^*}{2 \cdot |A^*| \cdot |B^*|} = s^*/2 \ , \tag{29}$$

by Equation (26),

$$\sum_{(a,b) \in P^*} e^*(a,b) \geq \frac{\epsilon_3 m}{2 \log^4 n} \ . \tag{30}$$

By Equation (30) and the premise of the claim regarding D, if we select an edge according to D, then the probability that we obtain an edge in $E^*(a,b)$ for $(a,b) \in P^*$ is $\Omega(\epsilon_3 / \log^4 n)$. Fixing any such pair (a,b), we slightly modify the definition of the random variables $\chi_{i,j}(a,b)$ so that $\chi_{i,j}(a,b) = 1$ if and only if the ith random sampled neighbor of a and the jth random sampled neighbor of b are both a common neighbor $c \in C^*(a,b)$ (the original definition allowed any common neighbor c (other than v_g)). We next upper bound the probability that $\sum_{i,j \in [\theta_1]} \chi_{i,j}(a,b) = 0$. Let

$$\mu(a,b) \stackrel{\text{def}}{=} \sum_{c \in C^*(a,b)} \frac{e(a,c)}{d(a)} \cdot \frac{e(b,c)}{d(b)} \ , \tag{31}$$

so that $\mathbb{E}[\chi_{i,j}(a,b)] = \Pr[\chi_{i,j}(a,b) = 1] = \mu(a,b)$. Since $(a,b) \in P^*$, by Equations (26)–(28),

$$\mu(a,b) \geq \frac{e^*(a,b)}{d(a) \cdot d(b)} \geq \frac{\epsilon_3 m}{2 \log^4 n \cdot d(a) \cdot d(b) \cdot |A^*| \cdot |B^*|} \geq \frac{\epsilon_3}{8 \log^4 n \cdot m} \ , \tag{32}$$

where we have used the fact that $d(a) \cdot |A^*| \leq 2m' \leq 2m$ and $d(b) \cdot |B^*| \leq 2m' \leq 2m$. From this point on we shall use the shorthand $\chi_{i,j}$ for $\chi_{i,j}(a,b)$ and μ for $\mu(a,b)$.

At this point we would have liked to apply Corollary 16 to upper bound the probability that $\sum_{i,j \in [\theta_1]} \chi_{i,j} = 0$, but it does not exactly meet our needs, and hence we give a direct proof. First we observe that

$$\mathbb{E}\left[\sum_{i,j \in [\theta_1]} \chi_{i,j} \right] = \theta_1^2 \cdot \mu \ . \tag{33}$$

By applying Claim 15 (see Appendix G), with $s_1 = s_2 = \theta_1$ and using the notation $\mu_{1,2} = \Pr[\chi_{i,j_1} = \chi_{i,j_2} = 1]$ (for every $i, j_1, j_1 \in [\theta_1]$) and $\mu_{2,1} = \Pr[\chi_{i_1,j} = \chi_{i_2,j} = 1]$ (for every $i_1, i_2, j \in [\theta_1]$) from that claim, we have that

$$\mathbb{V}\left[\sum_{i,j \in [\theta_1]} \chi_{i,j} \right] \leq \theta_1^2 \cdot \mu + \theta_1^3 \cdot \mu_{1,2} + \theta_1^3 \cdot \mu_{2,1} + \binom{\theta_1}{2}^2 \cdot \mu^2 - (\theta_1^2 \cdot \mu)^2 \ . \tag{34}$$

By Chebishev's inequality and the above two equations,

$$\Pr\left[\sum_{i,j \in [\theta_1]} \chi_{i,j} = 0 \right] \leq \frac{\mathbb{V}\left[\sum_{i,j \in [\theta_1]} \chi_{i,j}\right]}{\left(\mathbb{E}\left[\sum_{i,j \in [\theta_1]} \chi_{i,j}\right]\right)^2} \leq \frac{1}{\theta_1^2 \mu} + \frac{\mu_{1,2}}{\theta_1 \cdot \mu^2} + \frac{\mu_{2,1}}{\theta_1 \cdot \mu^2} \ . \tag{35}$$

We next upper bound each of the three terms on the right-hand-side of Equation (35).

By Equation (32) and the setting of $\theta_1 = c_1 \cdot n^{1/2} \log^2 n/\epsilon^2$, the first term contributes at most 1/6 (for a sufficiently large constant c_1).

Turning to the second term,

$$\mu_{1,2} = \Pr[\chi_{i,j_1} = \chi_{i,j_2} = 1] = \sum_{c \in C^*(a,b)} \frac{e(a,c) \cdot (e(b,c))^2}{d(a) \cdot (d(b))^2} \ . \tag{36}$$

By the definition of μ in Equation (31) and again using Equations (26)–(28), the second term is upper bounded by

$$\begin{aligned}
\frac{1}{\theta_1} \cdot \frac{d(a) \cdot \sum_{c \in C^*(a,b)} e(a,c) \cdot (e(b,c))^2}{\left(\sum_{c \in C^*(a,b)} e(a,c) \cdot e(b,c)\right)^2} &\leq \frac{1}{\theta_1} \cdot \frac{d(a) \cdot |C^*(a,b)| \cdot 2^{y^*_{A,C}} \cdot 2^{2y^*_{B,C}}}{|C^*(a,b)|^2 \cdot 2^{2(y^*_{A,C}-1)} \cdot 2^{2(y^*_{B,C}-1)}} && (37)\\
&= \frac{1}{\theta_1} \cdot \frac{16 d(a)}{|C^*(a,b)| \cdot 2^{y^*_{A,C}}} && (38)\\
&\leq \frac{1}{\theta_1} \cdot \frac{16 d(a)}{e^*(a,b)} && (39)\\
&\leq \frac{1}{\theta_1} \cdot \frac{8 \log^4 n \cdot d(a) \cdot |A^*| \cdot |B^*|}{\epsilon_3 m} && (40)\\
&\leq \frac{16 \log^4 n \cdot |B^*|}{\epsilon_3 \theta_1} \ . && (41)
\end{aligned}$$

Since $|B^*| \leq 2m/\theta_1$ (as $B^* \subseteq H$), by the setting of θ_1, the contribution of this term is at most 1/6 as well.

For the third term we have that

$$\mu_{2,1} = \Pr[\chi_{i_1,j_1} = \chi_{i_1,j_2}] = \sum_{c \in C^*(a,b)} \frac{(e(a,c))^2 \cdot e(b,c)}{d(a) \cdot d(b)} \ . \tag{42}$$

By essentially the same argument as the one bounding the second term (replacing the roles of a and b (A^* and B^*), this term contributes at most 1/6 as well.

We have thus shown that, conditioned on selecting an edge between a pair $(a,b) \in P^*$, the probability that we obtain a triangle is at least 1/2, and Claim 10 follows. □

In order to complete the proof of Claim 9 by applying Claim 10, we make the following observations. First, by the construction of G', each vertex in L' corresponds to exactly one blue edge in G'. In particular, if we select a vertex in L' and query all its neighbors (as done in the algorithm), then we effectively obtain an edge in G'. Since $|\mathcal{C}'_3| = \Omega(\epsilon^2 m)$, we have that $|L'| = \Omega(\epsilon^2 n)$. Therefore, the probability in each iteration that the algorithm selects a vertex in L', and hence an edge in G', is $\Omega(\epsilon^2)$.

Next, consider each of the endpoints u of such an edge. If $d(u) \leq \theta_1$, then our algorithm queries all the neighbors of u, and for each neighbor v of u such that $d(v) \leq \theta_0$, the algorithm queries all the neighbors of v. This implies that the algorithm reaches all of the neighbors $u' \in M'$ of u in G' (as in the corresponding case described in Claim 10). Similarly, if $d(u) > \theta_1$, then our algorithms samples θ_1 random neighbors of u, and for each sampled neighbor v such that $d(v) \leq \theta_0$, it queries all the neighbors of v. This implies that for every $u' \in M'$, the probability that it is reached from u by such a length-2 path in G, is at least as large as the probability that it is reached in one random step from u in G'. Since blue triangles in G' correspond to C_6s in G, Claim 9 follows. □

The proof of Theorem 11 follows directly from Claims 7–9 (together with the description of the algorithm for obtaining the upper bound on the complexity of the algorithm).

E A lower bound of $\Omega(n^{1/3})$ for all constant-length cycles

For the ease of readability, we first restate Theorem 2

Theorem 2 Let $k \geq 6$. Any one-sided error tester for the property of C_k-freeness in graphs of constant arboricity over n vertices must perform $\Omega(n^{1/3})$ queries.

As noted in the introduction, the lower bound stated in Theorem 2 also applies to graphs with non-constant arboricity by adding a C_k-free subgraph with higher arboricity. For an odd k, it suffices to add a dense bipartite graph, and for even k, by the Erdős girth conjecture [16], one can add a subgraph with arboricity $n^{2/k}$.

To prove the theorem we shall reduce (in a non black-box manner) from the following result:

Theorem 12 ([3, Lemma 2]) Every one-sided error algorithm for testing triangle-freeness in graphs with n' vertices and average degree d must perform $\Omega(d, n'/d)$ queries. This lower bound holds even when the maximum degree is $O(d)$.

Lemma 11 (Implicit in the proof of Lemma 2 in [3]) There exists a family of (multi-)graphs $\mathcal{G}_{n'}$ such that the following holds.

- The graphs are tripartite and have n' vertices in each part
- The graphs are d-regular for $d = \Theta(\sqrt{n'})$,
- All but at most a small constant fraction of graphs in the family are $\Omega(1)$-far from triangle-free.

Every algorithm that uses neighbor and/or vertex-pair queries on a graph selected uniformly at random from $\mathcal{G}_{n'}$ must perform $\Omega(\sqrt{n'})$ queries before it views a triangle with sufficiently high constant probability. Furthermore, this holds even when the algorithm knows in advance which vertex belongs to which of the three parts.

Proof of Theorem 2: To prove the theorem, assume towards a contradiction that there exists a one-sided error algorithm $\mathcal{A}$ for testing C_k-freeness using at most $n^{1/3}/c$ queries (for some constant c) in graphs with n vertices and arboricity 2. We shall prove that there exists an algorithm $\mathcal{A}'$ that finds, with high constant probability, a triangle in graphs selected uniformly from $\mathcal{G}_{n'}$ using at most $\sqrt{n'}/c'$ queries for some constant c', thus reaching a contradiction to Lemma 11.

Algorithm $\mathcal{A}'$ will operate as follows. Given query access to a graph $G' \in \mathcal{G}_{n'}$, algorithm $\mathcal{A}'$ (implicitly) defines a graph G and invokes algorithm $\mathcal{A}$ on it, where each query to G is answered using a constant number of queries to G' (in expectation). Once $\mathcal{A}$ terminates its execution on G, if it found a C_k in G, then $\mathcal{A}'$ returns a corresponding triangle in G', where there is a small probability ($o(1)$) that $\mathcal{A}'$ terminates early (before $\mathcal{A}$ makes its decision).

We shall show that if G' is ϵ'-far from triangle-free, then G is $\epsilon = \Omega(\epsilon')$-far from C_k-free. In such a case (when executed with a distance parameter ϵ), $\mathcal{A}$ must reject G with high constant probability. Since $\mathcal{A}$ has one-sided error, when it rejects, it must have evidence in the form of a C_k.[18]

For every graph G' in $\mathcal{G}_{n'}$, we consider an orientation of its edges so that for edge (u, v) if $id(u) < id(v)$, then the edge is oriented from u to v. For ease of presentation, assume for now that k is divisible by 3. The graph G is constructed in two steps where the first is deterministic and the second is randomized. First a graph G^+ is obtained by replacing each oriented edge $e = (u, u')$ in G' with a path of length $k/3$, where the original endpoints are kept in G^+, and $\ell = k/3 - 1$ new vertices $v_e^1, \ldots, v_e^\ell$ are added along the path (where $u = v_e^0$ and $u' = v_e^{\ell+1}$). We refer to the set of vertices in G^+ that originated from endpoints of edges in G' as "original vertices", and to the set of new vertices as "path vertices". See Figure 4 for an illustration. (Note that even if the graph G' has parallel edge, since every edge in G' is replaced by a distinct path in G^+, the resulting graph G^+ has no parallel edges.)

In the second step, the graph G is obtained by applying a random permutation on the ids of the vertices of G^+, where the permutation is applied separately to the ids of the original vertices and to the ids of the

[18] This statement holds for the graphs constructed, and assuming the number of queries performed by $\mathcal{A}$ is indeed at most $n^{1/3}/c$. The reason is that if after performing these many queries the algorithm did not detect a C_k, then the algorithm's "knowledge graph" (which includes all queried edges as well as degrees) can be completed to a graph that is C_k-free.

path vertices. That is, the algorithm $\mathcal{A}'$ knows at the beginning of its execution, what is the set of ids of the original vertices and of path vertices. If k is not divisible by 3, then to obtain G^+ from G', a similar process to the above is done with either $\lceil k/3 \rceil, \lceil k/3 \rceil, \lfloor k/3 \rfloor$ or $\lceil k/3 \rceil, \lfloor k/3 \rfloor, \lfloor k/3 \rfloor$, where all edges between the same two parts in the original graph G', are replaced by a path of the same length in G^+.

Recall that all graphs G' in $\mathcal{G}_{n'}$ have $m' = \Theta(n' \cdot d) = \Theta({n'}^{3/2})$ edges so that G has $n = \Theta(k \cdot m') = \Theta({n'}^{3/2})$ vertices and $m = \Theta({n'}^{3/2})$ edges (recall that k is a constant). Also note that G has arboricity 2, as for any subgraph of G, its average degree is at most 2.

Observe that by the above construction, every triangle in G' is transformed into a k-cycle in G, and every k-cycle in G originates from a triangle in G'. Therefore, if a k-cycle is detected in G, then a triangle is detected in G'. We also claim that the distance is preserved (up to a constant factor), i.e., that $dist(G, C_k\text{-}free) = \Omega(dist(G', C_3\text{-}free))$. To verify this, let R be a minimum-size set of edges so that if we remove R from G, then it becomes C_k-free. Map each edge $e \in R$ that resides on some path in G to the edge $e' \in E(G')$ that this path replaced, and let R' be the set of edges that R maps to. Then $|R'| = |R|$, and removing R' from G' will make G' triangle-free.[19] Therefore,

$$dist(G', C_3\text{-}free) \leq \frac{|R'|}{m'} = \frac{|R|}{m'} \leq \frac{|R|}{m \cdot \lfloor 3/k \rfloor} = \Omega(dist(G, C_k\text{-}free)) \ , \tag{43}$$

as claimed.

Hence, given access to G' in $\mathcal{G}_{n'}$, we shall invoke $\mathcal{A}$ on G. If G' is ϵ'-far from being triangle-free (for some constant ϵ', as almost all graphs in $\mathcal{G}_{n'}$ are), then G is ϵ'/c'-far from being C_k-free for some constant c'. Therefore, by the assumption on $\mathcal{A}$, we can, with high constant probability, using at most $n^{1/3}/c \leq \sqrt{n'}/c'$ queries on G, obtain a triangle in G'. Thus we get a contradiction to Lemma 11.

It remains to show that the invocation of $\mathcal{A}$ on G can be simulated by giving $\mathcal{A}'$ query access to G', with the same order of query complexity.

To answer queries to G using queries to G', $\mathcal{A}'$ constructs uniform permutations $\sigma_1 \in \Pi_{n'}$ and $\sigma_2 \in \Pi_{n-n'}$ "on the fly", so that σ_1 maps original vertices in G to original vertices in G^+ and σ_2 maps path vertices in G to path vertices in G^+.

Let $I_{1,t} \subset [n']$ and $I_{2,t} \subset [n'+1, n]$ be the set of indices, for which σ_1 and σ_2, respectively, were not yet decided on by the tth query. Further let $\sigma(I_{1,t})$ and $\sigma(I_{2,t})$ be the set of indices in $[n']$ and $[n'+1, n]$, respectively, that were not yet mapped to by σ by the tth query. For any $t \geq 0$, the $t+1$ query is answered by $\mathcal{A}$ as follows.

Degree queries $deg(u)$: Recall that every $G' \in \mathcal{G}_{n'}$ is d-regular, so that all the original vertices in G have degree d. Also, by construction, all path vertices in G have degree exactly 2. Finally, by construction, the set of ids of each group of vertices is known in advance. Hence, we can assume without loss of generality that no degree queries are performed.

Neighbor queries $nbr(u, p)$: The query is answered differently depending on whether $u \in [n']$ or $u \in [n'+1, n]$. If $u \in [n']$, then $\mathcal{A}'$ first selects $\sigma_1(u)$ by choosing an index in $[n'] \setminus \sigma_1(I_t)$ u.a.r.. Then $\mathcal{A}'$ performs a neighbor query on the original vertex $\sigma(u)$ in G'. That is, $\mathcal{A}'$ performs the query $nbr(\sigma(u), p)$ on G'. Let $e' = (\sigma(u), v)$ denote the returned edge. If v was not previously discovered (i.e., was not previously queried or was returned as an answer to a previous query), then $\mathcal{A}'$ decides on its id in G , i.e., $\sigma_1^{-1}(v)$, by choosing uniformly from the indices in $[n'] \setminus I_{1,t}$. $\mathcal{A}'$ also performs a path labeling process on the path that replaces the edge e' in G: Let $v^1_{e'}, \ldots, v^\ell_{e'}$ be the path vertices in G^+ that on the path that replaced the edge e' in G' (where $\sigma(u) = v^0_{e'}$ and $v = v^{\ell+1}_{e'}$). $\mathcal{A}'$ labels the path vertices by choosing for each $v^r_{e'}$ a value $\sigma_2^{-1}(v^r_{e'})$ u.a.r. in $[n'+1, n] \setminus I_{2,t}$. Finally, $\mathcal{A}'$ answers the query with all the vertices of the path and their ids. We refer to this as a "path answer".

[19] Assume the contrary, i.e., that after removing R' from G', there exists a triangle in G'. Then the three paths in G that replaced this triangle are disjoint from R, as otherwise this path would not have survived. Therefore, G after removing R is not C_k-free which is a contradiction to the definition of R.

We turn to the case that the neighbor query is performed on a path vertex. That is, $u \in [n'+1, n]$. In this case we would like to map $u \in G$ to an edge $e' \in G'$. Recall that the id u not only specifies that u is a path vertex, but also to which two parts its path belongs to. Assume without loss of generality that u belongs to the set of path vertices that reside on paths between parts 1 and 2. In order to map u to an edge between parts P_1, P_2, we need to first sample an edge e between P_1 and P_2 uniformly at random (we shortly explain how). If the edge $e' \in G'$ has already been discovered, then $\mathcal{A}'$ terminates and returns that G' is triangle free. This only happens with negligible probability $O(Q/m) = o(1)$, where Q is the number of queries performed by $\mathcal{A}$. Otherwise, $\mathcal{A}'$ draws a uniform index $r \in [\ell]$, and lets $\sigma(u) = v^r_{e'}$. Then $\mathcal{A}'$ performs a path labeling process on the remaining path vertices and returns a path answer for e'.

To sample a uniform edge in G' between two specific parts, say P_1 and P_2, $\mathcal{A}'$ samples w in the set of indices of P_1 and $r' \in [d]$ u.a.r. and performs neighbor queries $nbr(w, r')$ on G' until the returned neighbor belongs to the desired part, in this case P_2. Since G' is d-regular, this results in a uniformly distributed edge e' in $E(P_1, P_2)$. Note that since every vertex $v \in G$ has exactly half of its neighbors in each of the other two parts, the above process returns an edge in the desired part with probability $1/2$. Hence, the expected number of queries per one query simulation is constant.

Pair queries $pair(u_1, u_2)$: If the two queried vertices are original vertices in G' then the answer to the query is no. If at least one of them is a path vertex, then $\mathcal{A}$ maps it to an edge, as described above for neighbor queries, and answers accordingly.

Finally, as each query to G can be simulated in a constant number of queries to G' in expectation, it follows that with high probability $1 - o(1)$ (over the randomness of $\mathcal{A}'$), the total number of queries does not exceed that of $\mathcal{A}$ by more than a constant factor, and otherwise $\mathcal{A}'$ may terminate. This concludes the proof. □

F A general upper bound for testing C_k-freeness (and F-freeness in general)

In this we section describe and analyze our algorithms for testing C_k-freeness, for even and odd k, and more generally for F-freeness, for any fixed subgraph F. We shall use k to denote the number of vertices in F (which is consistent with the special case that $F = C_k$). In order to present the result for F-freeness, we introduce a measure that extends the notion of the size of a minimum vertex cover.

Definition 1 For a graph $F = (V_F, E_F)$ let $\mathcal{VC}(F)$ denote the set of all vertex covers of F. For a vertex cover Z of F we denote by $\mathcal{VC}'(Z)$ the set of vertex covers of F that are subsets of Z. We define $\ell(F) = \max_{Z \in \mathcal{VC}(F)} \left\{ min_{B \in \mathcal{VC}'(Z)} (|B|) \right\}$.

Observe that by Definition 1, we have that $\ell(F)$ is lower bounded by the size of a minimum vertex cover of F and is upper bounded by $k = |V_F|$.

Our algorithm for testing F-freeness appears next. The algorithm receives the number of edges, m, as a parameter. We note that m can be an upper bound on the number of edges of the graph. The algorithm also receives α as a parameter. As noted before, if m is given (and not just an upper bound) then we can estimate the effective arboricity instead of receiving α as a parameter. Another option is to receive only α (without m) and use αn as an upper bound on m.

Test-subgraph-freeness$(F, n, m, \alpha, \epsilon)$

1. Set $\theta_0 = 4\alpha/\epsilon$.
2. Sample u.a.r. $s = \Theta\left(k^{2+1/\ell(F)} \cdot m \cdot \left(\frac{\alpha}{m}\right)^{1/\ell(F)} \cdot \left(\frac{1}{\epsilon}\right)^{1+2/\ell(F)}\right)$ vertices from G (where $\ell(F)$ is as defined in Definition 1).

3. Query all the neighbors of every vertex v in the sample such that $d(v) \leq \theta_0$.

4. Reject if and only there is a witness for F in the resulting subgraph.

In order to analyze the algorithm Test-subgraph-freeness, we introduce some definitions, and prove a claim regarding sampling of tuples.

For the threshold θ_0 as defined in the algorithm, let L denote the subset of vertices in G whose degree is at most θ_0, and let H denote the subset of vertices whose degree is greater than θ_0. We refer to the former as light vertices and the latter as heavy vertices. We say that a subset of subgraphs of G (in particular, that are isomorphic of F), are light-vertex-disjoint if they do not share any light vertex. Let $G_{\leq \theta_0}$ denote the subgraph of G that contains all vertices of G but only edges that have at least one light endpoint.

Claim 12 Let X be a set of elements and let $\mathcal{T} = \{(x_1^j, \ldots, x_\ell^j)\}_{j \in [|\mathcal{T}|]}$ be a set of ℓ-tuples of elements of X such that each $x \in X$ appears at most once in some $\rho \in \mathcal{T}$ (namely, the ℓ-tuples are disjoint and each tuple contains distinct elements). Let $S = \{u_1, \ldots u_s\}$ be a multi-set of s elements chosen independently, uniformly, at random from X. If $s \geq 16\ell|X|/(|\mathcal{T}|^{1/\ell})$, then with high constant probability over the choice of S, it contains ℓ elements that belong to a common tuple in $\mathcal{T}$.

Proof: We say that an ℓ-tuple of indices $(i_1, \ldots i_\ell)$ where $i_j \in [s]$ for each $j \in [\ell]$ is good with respect to S and $\mathcal{T}$ if $(u_{i_1}, \ldots u_{i_\ell})$ belongs to $\mathcal{T}$. Let $r = \binom{s}{\ell} \cdot \ell!$ denote the number of different ℓ-tuples of indices in $[s]$. Consider any fixed ordering of these tuples, and let χ_i be the indicator variable for the event that the i-th tuple of indices is good with respect to S and $\mathcal{T}$. We next upper bound the probability that $\sum_{i \in [r]} \chi_i = 0$. Let $\mu = \mathbb{E}[\chi_i]$ and $\bar{\chi}_i = \chi_i - \mu$, so that $\mathbb{E}[\bar{\chi}_i] = 0$. By Chebyshev's inequality,

$$\Pr\left[\sum_{i \in [r]} \chi_i = 0\right] \leq \frac{\mathbb{V}\left[\sum_{i \in [r]} \chi_i\right]}{\left(\mathbb{E}\left[\sum_{i \in [r]} \chi_i\right]\right)^2} = \frac{\sum_{i,j} \mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j]}{r^2 \cdot \mu^2} \ . \tag{44}$$

We first calculate:

$$\mathbb{E}\left[\sum_{i \in [r]} \chi_i\right] = r \cdot \mu = \binom{s}{\ell} \cdot \ell! \cdot |\mathcal{T}| \cdot \left(\frac{1}{|X|}\right)^\ell = \frac{s!}{(s-\ell)!} \cdot |\mathcal{T}| \cdot \left(\frac{1}{|X|}\right)^\ell \geq (s/2)^\ell \cdot |\mathcal{T}| \cdot \left(\frac{1}{|X|}\right)^\ell \geq (8\ell)^\ell \ . \tag{45}$$

We next break the sum $\sum_{i,j} \mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j]$ into several sub-sums as follows. We say that $(i,j) \in [r] \times [r]$ is in R_t if the number of elements (namely, indexes of samples) that belong both to the i-th tuple and the j-th tuple is exactly t (namely the intersection of the tuples is of size t when they are viewed as sets).

If $(i,j) \in R_0$ then χ_i and χ_j are independent and so

$$\mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j] = \mathbb{E}[\bar{\chi}_i] \cdot \mathbb{E}[\bar{\chi}_j] = 0 \ . \tag{46}$$

Hence, the pairs in R_0 do not contribute anything to the right-hand side of Equation (44).

If $i = j$ then

$$\mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j] = \mathbb{E}[(\bar{\chi}_i)^2] \leq \mathbb{E}[(\chi_i)^2] = \mathbb{E}[\chi_i] = \mu \ , \tag{47}$$

where the second inequality follows from the definition of $\bar{\chi}_i$.

Given that $(i,j) \in R_t$ it holds that

$$\mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j] \leq \mathbb{E}[\chi_i \chi_j] \leq |\mathcal{T}| \cdot \left(\frac{1}{|X|}\right)^{2\ell - t} \ . \tag{48}$$

Since there are at most $\binom{\ell}{t} \cdot s^{2\ell-t}$ pairs in R_t we obtain that

$$\begin{aligned}\sum_{(i,j)\in R_t} \mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j] &\leq \binom{\ell}{t} \cdot s^{2\ell-t} \cdot |\mathcal{T}| \cdot \left(\frac{1}{|X|}\right)^{2\ell-t} \leq \ell^t \cdot |\mathcal{T}| \cdot \left(\frac{s}{|X|}\right)^{2\ell-t} \leq \ell^\ell \cdot |\mathcal{T}| \cdot \left(\frac{s}{|X|}\right)^{\ell} \\ &= \left(\frac{|\mathcal{T}|^{1/\ell} \cdot \ell \cdot s}{|X|}\right)^{\ell} \leq (16\ell^2)^\ell \,, \end{aligned} \tag{49}$$

Thus,

$$\frac{\sum_{i,j} \mathbb{E}[\bar{\chi}_i \cdot \bar{\chi}_j]}{r^2 \cdot \mu^2} \leq \frac{1}{r\mu} + \frac{\ell(16\ell^2)^\ell}{(r\mu)^2} \leq \frac{1}{(8\ell)^\ell} + \frac{\ell(16\ell^2)^\ell}{(8\ell)^{2\ell}} \leq \frac{1}{8} + \frac{\ell}{2^{2\ell}} \leq \frac{3}{8} \,, \tag{50}$$

which concludes the proof of the claim. □

Theorem 13 The algorithm *Test-subgraph-freeness* is a one-sided error tester for F-freeness whose query complexity is

$$O\left(k^{2+1/\ell(F)} \cdot m \cdot \left(\frac{\alpha}{m}\right)^{1/\ell(F)} \cdot \left(\frac{1}{\epsilon}\right)^{1+2/\ell(F)}\right) .$$

Proof: The completeness of the tester follows by construction. We next prove its soundness (where we start similarly to previous proofs). Assume G is ϵ-far from being F-free. By the setting of $\theta_0 = 4\alpha/\epsilon$, the graph $G_{\leq\theta_0}$ is $\epsilon/2$-far from being F-free. By the definition of $G_{\leq\theta_0}$, every copy of F in $G_{\leq\theta_0}$ (i.e., subgraph of $G_{\leq\theta_0}$ that is isomorphic to F) only contains edges that are incident to light vertices. Consider an iterative process, that constructs a set $\mathcal{F}$ of light-vertex-disjoint copies of F in $G_{\leq\theta_0}$ in the following greedy manner. In each iteration it selects a new copy of F in $G_{\leq\theta_0}$ that does not share any light vertices with previously selected copies. It follows that $|\mathcal{F}| \geq \epsilon m/(2\theta_0 k)$ (to see this consider a greedy process which removes all the edges that are incident to the light vertices of previously selected copies).

From each one of these copies we pick a subset of light vertices that covers the edges of this copy (since the copy is in $G_{\leq\theta_0}$ there exists a vertex cover which is composed of only light vertices). Moreover, by the definition of $\ell(F)$, it suffices to pick $\ell(F)$ light vertices for each copy. By ordering these vertices (arbitrarily) we obtain an $\ell(F)$-tuple of light vertices for each one of these copies. Let $\mathcal{T}$ to be the set of $\ell(F)$-tuples corresponding to these copies. According to Claim 12, if we set s to be at least $16\ell(F)n/((\epsilon m/(2\theta_0 k))^{1/\ell(F)})$ vertices, then w.h.c.p. there exists a copy of F in G such that we hit all its light vertices. Conditioned on this event, since the algorithm queries all the neighbors of all light vertices in the sample, the copy will be revealed. By Claim 14, w.h.p., the number of queries the algorithm needs to preform is bounded by $s \cdot 2\bar{d} = O\left(k^{2+1/\ell(F)} \cdot m \cdot \left(\frac{\alpha}{m}\right)^{1/\ell(F)} \cdot \left(\frac{1}{\epsilon}\right)^{1+2/\ell(F)}\right)$. If the number of queries exceeds this bound (and the algorithm did not find a witness for rejection) the algorithm can simply accept. □

Corollary 14 There exists a one-sided error tester for C_k-freeness whose query complexity is

$$O\left(k^{2+(2/k))} \cdot m \cdot \left(\frac{\alpha}{m}\right)^{2/k} \cdot \left(\frac{1}{\epsilon}\right)^{1+(4/k)}\right)$$

for even k, and

$$O\left(k^{2+2/(k+1)} \cdot m \cdot \left(\frac{\alpha}{m}\right)^{2/(k+1)} \cdot \left(\frac{1}{\epsilon}\right)^{1+4/(k+1))}\right)$$

for odd k.

We next turn to describe and analyze an algorithm for testing C_k-freeness when k is odd, where the algorithm improves on the upper bound stated in Corollary 14 (when k is odd) for a certain range of values of α.

We first slightly modify the algorithm Select-an-edge from Section 3 to obtain an algorithm that samples a uniform edge from $G_{\leq\theta_0}$. As shown in [25], the expected running time of this algorithm is $O(\theta_0 n/m)$.

Sample a uniform edge from $G_{\leq\theta_0}$

- Repeat until an edge is returned:
  1. Sample u.a.r. a vertex v from $V(G)$.
  2. Pick $j \in [\theta_0]$ and toss a fair coin $b \in \{\text{Heads}, \text{Tails}\}$.
  3. If $v \in L$ and v has a j-th neighbour, u then:
     (a) If $u \in H$, then return $\{u, v\}$.
     (b) If $u \in L$ and $b = \text{Heads}$, then return $\{u, v\}$.

Claim 13 Let E' be a set of βm edges in $G_{\leq\theta_0}$, where $\beta \in (0, 1]$ and let m' denote the number of edges in $G_{\leq\theta_0}$. If we sample $\log n/\beta \leq y \leq m'/2$ edges uniformly from $G_{\leq\theta_0}$, then with high probability we sample at least $c\beta y$ different edges from E', where c is an absolute constant.

Proof: Let X_i denote the indicator variable for the event that the i-th sample is an edge from E'. Since the edges are sampled uniformly from $G_{\leq\theta_0}$, $\mathbb{E}(X_i) = \beta$. Thus, by multiplicative Chernoff's bound, with high probability $\sum_i X_i = b\beta y$, where b is a constant. Let Y_i be the indicator variable for the event that the i-th sample is a new edge from E' (with respect to the $(i-1)$ first samples). We consider, without loss of generality, the sampling process as if it first decides the values of X_i for each sample i and then samples the edges one by one accordingly. Consider the j-th sample. Given the values of X_i for every i and the values of the samples for each $i < j$, the probability that the j-th sample is a new edge, conditioned on the event that $X_j = 1$ is constant (since $y \leq x/2$). Therefore, with high probability we sample $c\beta y$ different edges from E' for some absolute constant c. □

Test-C_k-freeness when k is odd(n, m, α, ϵ)

1. Set $\theta_0 = 4\alpha/\epsilon$.
2. Sample u.a.r. $s_1 = \Theta\left(k \cdot m^{1-\frac{2}{k-1}} \cdot (1/\alpha)^{1-\frac{4}{k-1}} \cdot \epsilon^{1-\frac{6}{k-1}}\right)$ edges from $G_{\leq\theta_0}$ (as described above). For each edge in the sample, reveal the neighborhood of both endpoints if both are light.
3. If $k > 3$, then sample u.a.r. $s_2 = O\left(k \cdot n \cdot (\alpha^2/m)^{\frac{2}{k-1}} \cdot \left(\frac{1}{\epsilon}\right)^{\frac{6}{k-1}}\right)$ vertices and reveal the neighborhood of all the light vertices in the sample.
4. Reject iff there is a witness for C_k in the resulting subgraph.

The following bound is an improved bound for testing C_k-freeness for odd k whenever $m = \Omega(\alpha^{(k+3)/2})$ and in particular when α is a constant.

Theorem 15 There exists a one-sided error tester for C_k-freeness for odd k whose query complexity is

$$O\left(k \cdot m \cdot (\alpha^2/m)^{\frac{2}{k-1}} \cdot \left(\frac{1}{\epsilon}\right)^{\frac{6}{k-1}}\right)$$

Proof: The completeness of the tester follows by construction. We next prove its soundness. Assume G is ϵ-far from being C_k-free. As in the proof of Theorem 13, there exists a set $\mathcal{C}$ of light-vertex-disjoint copies of C_k in $G_{\leq\theta_0}$ such that $|\mathcal{C}| \geq \epsilon m/(2\theta_0 k)$. For each one of the copies in $\mathcal{C}$ we pick a subset of light vertices that covers the edges of this copy (since the copy is in $G_{\leq\theta_0}$ there exists a vertex cover which is composed of only light vertices). By the structure of C_k, it suffices to pick $\lceil k/2 \rceil$ light vertices from each copy. Moreover, exactly one pair of these vertices are on the same edge of the corresponding copy C_k. By excluding this pair and ordering the rest of these vertices (arbitrarily) , we obtain an $(k-3)/2$-tuple of light vertices for each one of these copies.

Let $\mathcal{T}$ denote the set of these tuples. We also consider the set of pairs of vertices we excluded from each copy. Each such pair is an edge of a C_k in $\mathcal{C}$. Denote this set of edges by Y, so that $|Y| \geq \epsilon m/(2\theta_0 k)$. By Claim 13, w.h.p., we sample at least $c\epsilon s_1/(2\theta_0 k)$ edges from Y (where c is a constant). Let $\mathcal{T}' \subseteq \mathcal{T}$ be the set of tuples corresponding to copies for which we sampled an edge from Y. By the above, w.h.p., $|\mathcal{T}'| \geq c\epsilon s_1/(2\theta_0 k)$. Conditioning on this event, according to Claim 12, if s_2 is at least $16kn/(c\epsilon s_1/(2\theta_0 k))^{2/(k-3)}$, then w.h.c.p. the algorithm will find a copy of C_k. By Claim 14 the expected query complexity of the algorithm is bounded by $O(s_1 \cdot \theta_0 + s_2 \cdot \bar{d}) = O\left(k \cdot m^{1-\frac{2}{k-1}} \cdot \alpha^{\frac{4}{k-1}} \cdot \left(\frac{1}{\epsilon}\right)^{\frac{6}{k-1}}\right)$. Thus, by Markov's inequality, if the number of queries exceeds this bound (and the algorithm did not find a witness for rejection) the algorithm can simply accept. □

Claim 14 Let $\bar{d}$ be the average degree in G, and let $S = \{v_1, \ldots, v_s\}$ be a uniform sample of size at least $s = (\theta_0/\bar{d}) \cdot 12\ln(1/\delta)$ vertices (with repetitions). Let $\chi_i = d(v_i)$ if $d(v_i) \leq \theta_0$ and $\chi_i = 0$ otherwise. Then with probability at least $1-\delta$, $\frac{1}{s}\sum_{i=1}^{s} \chi_i \leq 2\bar{d}$.

Proof: It holds that $\mathbb{E}[\chi_i] \leq \frac{1}{n}\sum_{v\in V} d(v) = \bar{d}$. For any $\epsilon < \frac{1}{2}$, $\mathbb{E}[\chi_i] \geq \frac{1}{n}\sum_{v:d(v)\leq t} d(v) \geq \frac{1}{n} \cdot 2(1-\epsilon)m \geq \bar{d}$.

Also, for every $i \in [s]$, $\chi_i \in [0, \theta_0]$ for every i. Let $x_i = \chi/\theta_0$ and $X = \sum_{i=1}^{s} x_i$. Then the x_i's are bounded independent random variables in $[0,1]$ with $\mathbb{E}[X] \leq s \cdot \bar{d}/\theta_0$. Therefore, by a coupling argument and the multiplicative Chernoff's bound

$$\Pr\left[X > 2s \cdot \frac{\bar{d}}{\theta_0}\right] \leq \exp\left(-s \cdot \bar{d}/3\theta_0\right) < \delta.$$

□

G Some probabilistic claims

Claim 15 For integers s_1 and s_2 let $\{\chi_{i,j}\}_{(i,j)\in[s_1]\times[s_2]}$ be Bernoulli random variables for which the following holds.

1. $\Pr[\chi_{i,j} = 1] = \mu$ for every $(i,j) \in [s_1] \times [s_2]$.
2. For every $i_1, i_2 \in [s_1]$, $j_1, j_2 \in [s_2]$ such that $i_1 \neq i_2$ and $j_1 \neq j_2$, the random variables χ_{i_1,j_2} and χ_{i_2,j_2} are independent.
3. For every $i \in [s_1]$, $j_1, j_2 \in [s_2]$ such that $j_1 \neq j_2$, $\Pr[\chi_{i,j_1} = \chi_{i,j_2} = 1] = \mu_{1,2}$, and for every $i_1, i_2 \in [s_1]$ and $j \in [s_2]$ such that $i_1 \neq i_2$, $\Pr[\chi_{i_1,j} = \chi_{i_2,j} = 1] = \mu_{2,1}$.

Then

$$\mathbb{V}\left[\sum_{i\in[s_1],j\in[s_2]} \chi_{i,j}\right] \leq s_1 \cdot s_2 \cdot \mu + s_1 \cdot \binom{s_2}{2} \cdot \mu_{1,2} + \binom{s_1}{2} \cdot s_2 \cdot \mu_{2,1} \,.$$

Proof: For each pair $(i,j) \in [s_1] \times [s_2]$, let $\bar{\chi}_{i,j} = \chi_{i,j} - \mu$, so that $\bar{\chi}_{i,j} \leq \chi_{i,j}$, and $\mathbb{E}[\bar{\chi}_{i,j}] = 0$. By the definition of the variance,

$$\mathbb{V}\left[\sum_{i\in[s_1],j\in[s_2]} \chi_{i,j}\right] = \sum_{(i_1,j_2),(i_2,j_2)\in[s_1]\times[s_2]} \mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_2,j_2}] \,. \tag{51}$$

We upper bound the right-hand-side of Equation (51) by breaking the sum $\sum_{i_1,j_2,i_2,j_2} \mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_2,j_2}]$ into four sub-sums.

The first is over $(i_1,j_1),(i_2,j_2) \in [s_1] \times [s_2]$ such that all four indices are distinct. By the second condition in the claim, for such quadruples of indices, χ_{i_1,j_1} is independent of χ_{i_2,j_2} and so

$$\mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_2,j_2}] = \mathbb{E}[\bar{\chi}_{i_1,j_1}] \cdot \mathbb{E}[\bar{\chi}_{i_2,j_2}] = 0 \,. \tag{52}$$

The second is over $i_1 = i_2$ and $j_1 = j_2$, so that

$$\mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_2,j_2}] = \mathbb{E}[(\bar{\chi}_{i_1,j_1})^2] \leq \mathbb{E}[(\chi_{i_1,j_1})^2] = \mathbb{E}[\chi_{i_1,j_1}] = \mu \,, \tag{53}$$

where the second inequality follows from the definition of $\bar{\chi}_{i,j}$. There are $s_1 \times s_2$ such quadruples.

The third is over $i_1 = i_2$ and $j_1 \neq j_2$, in which case we have

$$\mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_1,j_2}] \leq \mathbb{E}[\chi_{i_1,j_1} \cdot \chi_{i_1,j_2}] = \Pr[\chi_{i_1,j_1} = \chi_{i_1,j_2} = 1] = \mu_{1,2} \,. \tag{54}$$

There are $s_1 \times \binom{s_2}{2}$ such pairs.

The fourth is over $i_1 \neq i_2$ and $j_1 = j_2$, where similarly to the third case we get

$$\mathbb{E}[\bar{\chi}_{i_1,j_1} \cdot \bar{\chi}_{i_2,j_2}] \leq \mu_{2,1} \tag{55}$$

where there are $\binom{s_1}{2} \cdot s_2$ such pairs.

The claim follows by summing all four sub-sums. □

The next claim is very similar to Claim 15, but the setting is slightly different, and will be used in our analysis several time. In what follows, for an integer s we use $\Phi(s)$ to denote the set of pairs $i,j \in [s]$ such that $i < j$.

Claim 16 For an integer sl let $\{\chi_{i,j}\}_{(i,j)\in\Phi(s)}$ be Bernoulli random variables for which the following holds.

1. $\Pr[\chi_{i,j} = 1] = \mu$ for every $(i,j) \in \Phi(s)$.
2. For every $i_1,i_2 \in [s_1]$, $j_1,j_2 \in [s_2]$ such that $i_1 \neq i_2$ and $j_1 \neq j_2$, the random variables χ_{i_1,j_2} and χ_{i_2,j_2} are independent.
3. For every $(i_1,j_1) \in \Phi(s)$ and $(i_2,j_2) \in \Phi(s)$ such that exactly two of the four indices are the same, $\Pr[\chi_{i_1,j_1} = \chi_{i_2,j_2} = 1] = \mu_{1,2}$.

Then $\mathbb{V}\left[\sum_{(i,j)\in\Phi(s)} \chi_{i,j}\right] \leq \binom{s}{2} \cdot \mu + \binom{s}{3}\mu_{1,2}$.

The proof of Claim 16 is very similar to the proof of Claim 15, and is hence omitted. As a corollary of this claim we get:

Corollary 16 For an integer s let $\{\chi_{i,j}\}_{(i,j)\in\Phi(s)}$ be Bernoulli random variables where $\Pr[\chi_{i,j} = 1] = \mu$ for every $(i,j) \in \Phi(s)$. Suppose that the following conditions hold for some $c_1 > 0$ and $c_2 > 4$.

1. For every $(i_1,j_1) \in \Phi(s)$ and $(i_2,j_2) \in \Phi(s)$ such that the four indices are distinct, χ_{i_1,j_1} and χ_{i_2,j_2} are independent.
2. For every $(i_1,j_1) \in \Phi(s)$ and $(i_2,j_2) \in \Phi(s)$ such that exactly two of the four indices are the same, $\Pr[\chi_{i_1,j_1} = \chi_{i_2,j_2} = 1] \leq c_1 \cdot \mu^{3/2}$.
3. $s \geq c_2/\sqrt{\mu}$.

Then $\Pr\left[\sum_{(i,j)\in\Phi(s)} \chi_{i,j} = 0\right] \leq \frac{1+c_1}{c_2}$.

Proof: By Chebishev's inequality,

$$\Pr\left[\sum_{(i,j)\in\Phi(s)} \chi_{i,j} = 0\right] \leq \frac{\mathbb{V}\left[\sum_{(i,j)\in\Phi(s)} \chi_{i,j}\right]}{\left(\mathbb{E}\left[\sum_{(i,j)\in\Phi(s)} \chi_{i,j}\right]\right)^2} \tag{56}$$

The denominator of the right-hand-side of Equation (56) equals $\binom{s}{2}^2 \cdot \mu^2$. By Claim 16 and the first two conditions of the current claim, the numerator of the right-hand-side of Equation (56) is upper bounded by $\binom{s}{2} \cdot \mu + \binom{s}{3} \cdot c_1 \cdot \mu^{3/2}$, so that the claim follows. □